UNIVERSIDADE D
COIMBRA

Filipe de Ornelas Diogo Obrist

TOWARDS THE NEUROEVOLUTION OF TRANSFORMER ARCHITECTURES

Dissertation in the context of the Master Data Science and Engineering, specialization in Neuroevolution, advised by Professor João Nuno Gonçalves Costa Cavaleiro Correia and Professor Nuno António Marques Lourenço and presented to the Department of Informatics Engineering of the Faculty of Sciences and Technology of the University of Coimbra.

July 2026

DEPARTAMENTO DE
ENGENHARIA INFORMÁTICA
FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

Filipe de Ornelas Diogo Obrist

TOWARDS THE NEUROEVOLUTION OF TRANSFORMER ARCHITECTURES

Dissertation in the context of the Master Data Science and Engineering, specialization in Neuroevolution, advised by Professor João Nuno Gonçalves Costa Cavaleiro Correia and Professor Nuno António Marques Lourenço and presented to the Department of Informatics Engineering of the Faculty of Sciences and Technology of the University of Coimbra.

July 2026

DEPARTAMENTO DE
ENGENHARIA INFORMÁTICA
FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

Filipe de Ornelas Diogo Obrist

Rumo à Neuroevolução de Arquiteturas Transformer

Dissertação no âmbito do Mestrado em Engenharia e Ciência de Dados, especialização em Neuroevolução, orientada pelo Professor João Nuno Gonçalves Costa Cavaleiro Correia e Professor Nuno António Marques Lourenço e apresentada ao Departamento de Engenharia Informática da Faculdade de Ciências e Tecnologia da Universidade de Coimbra.

Julho 2026

Statement on the use of artificial intelligence tools

This document was reviewed and refined with the assistance of Large Language Models, such as ChatGPT, or similar tools, which helped check grammar, correct typos, and enhance clarity. Any Artificial Intelligence-generated text or content is clearly indicated within the document and is used only to a limited extent. The overall content and ideas remain solely the responsibility of the author.

Abstract

Modern language models are overwhelmingly built on the Transformer architecture, whose self-attention mechanism, though powerful, incurs quadratic computational and memory costs. More recently, State Space Models such as Mamba have emerged as efficient alternatives with linear complexity, and hybrid architectures like Jamba have shown that the two paradigms can be combined. However, the design of these hybrids remains a manual, expert-driven process, and the potential of Evolutionary Computation to explore the vast combinatorial space of possible layer arrangements has not been investigated.

This dissertation presents an exploratory study of whether a simple genetic algorithm can automatically design hybrid Transformer–Mamba architectures for text classification. Using a variable-length binary genotype that specifies the type of each layer, the algorithm evolves populations of candidate architectures, evaluating their fitness through a short, budget-limited training. The best architectures are then further trained and compared against hand-crafted baselines.

Experiments on two distinct classification tasks reveal that the evolutionary search consistently converges to compact, Mamba-dominant architectures that are remarkably parameter-efficient: they match or surpass the performance of larger, manually designed hybrids while using significantly fewer parameters, and they approach the performance of heavily pre-trained Transformers, all while being trained from scratch on task-specific data.

Beyond the specific outcomes, this work demonstrates that Evolutionary Computation can serve as a practical tool for architectural exploration even at the scale of modern language models. It establishes a first baseline at the intersection of neuroevolution and hybrid sequence models Transformer-Mamba, offering a launching zone for future research into automated architecture design for efficient language understanding.

Keywords

Neural Architecture Search, Evolutionary Computation, Transformers, Mamba, Hybrid Architectures, Large Language Models.

Resumo

Os modelos de linguagem modernos baseiam-se, na sua grande maioria, na arquitetura Transformer, cujo mecanismo de atenção própria, embora poderoso, acarreta custos computacionais e de memória que crescem quadraticamente. Mais recentemente, surgiram os Modelos de Espaço de Estados, como o Mamba, que oferecem complexidade linear e, com arquiteturas híbridas como o Jamba, mostraram que é possível combinar os dois paradigmas. No entanto, o desenho destes modelos híbridos continua a ser um processo manual, guiado pela intuição de especialistas, e o potencial da Computação Evolutiva para explorar o vasto espaço combinatório de combinações de camadas nunca foi investigado.

Esta dissertação apresenta um estudo exploratório sobre a possibilidade de um algoritmo genético simples conceber automaticamente arquiteturas híbridas Transformer–Mamba para classificação de texto. Recorrendo a um genótipo binário de comprimento variável que especifica o tipo de cada camada, o algoritmo evolui populações de arquiteturas candidatas, avaliando a sua aptidão através de um treino curto, com orçamento limitado. As melhores arquiteturas são depois treinadas de forma intensiva e comparadas com modelos de referência desenhados manualmente.

As experiências realizadas em duas tarefas de classificação distintas revelam que a procura evolutiva converge consistentemente para arquiteturas compactas e predominantemente Mamba, que se revelam notavelmente eficientes em termos de parâmetros: igualam ou superam o desempenho de modelos híbridos maiores, desenhados à mão, utilizando significativamente menos parâmetros, e aproximam-se do desempenho de Transformers fortemente pré-treinados, mesmo sendo treinadas de raiz apenas com os dados da tarefa.

Para além dos resultados concretos, este trabalho mostra que a Computação Evolutiva pode ser uma ferramenta prática para a exploração arquitetural, mesmo à escala dos modelos de linguagem atuais. Estabelece uma primeira referência na interseção entre neuroevolução e modelos de sequência híbridos Transformer-Mamba, criando uma plataforma de partida para investigação futura em conceção automatizada de arquiteturas para compreensão eficiente da linguagem.

Palavras-Chave

Pesquisa de Arquiteturas Neuronais, Computação Evolucionária, Transformers, Mamba, Arquiteturas Híbridas, Grandes Modelos de Linguagem

Contents

Acronyms

EAs Evolutionary algorithms.

EC Evolutionary Computation.

ENAS Evolutionary Neural Architecture Search.

ES Evolution Strategies.

GAs Genetic Algorithms.

GRU Gated Recurrent Unit.

LLM Large Language Model.

LSTM Long Short-Term Memory.

MoE Mixture of Experts.

NAS Neural Architecture Search.

NLP Natural Language Processing.

PBT Population Based Training.

RNNs Recurrent Neural Networks.

SSM State Space Model.

List of Figures

List of Tables

Chapter 1

Introduction

Large Language Models (LLMs) have become a central component of modern Artificial Intelligence, achieving remarkable performance across a wide range of natural language processing tasks, including text generation, translation, summarization, and scientific reasoning. These advances have been driven primarily by the Transformer architecture, whose self-attention mechanism enables effective modeling of long-range dependencies in sequential data [38]. As a consequence, most contemporary LLMs rely on increasingly deep and wide Transformer-based architectures, often comprising hundreds of millions or even billions of parameters [19, 38].

Despite their success, Transformer-based models present some challenges. Their reliance on global self-attention leads to quadratic computational and memory complexity with respect to sequence length, resulting in high training and inference costs [38]. These requirements limit accessibility, increase energy consumption, and raise concerns regarding scalability and sustainability. Furthermore, the design of Transformer architectures remains largely manual, relying on expert intuition and incremental refinements of existing models rather than a principled exploration of the architectural design space [2, 38]. This manual process risks converging prematurely on over-engineered or suboptimal solutions, particularly when adapting models to new domains or deployment constraints.

These computational and design limitations have motivated growing interest in automated approaches to architecture discovery. Evolutionary Neural Architecture Search (ENAS) and related approaches have demonstrated that Evolutionary Computation (EC) can successfully optimize neural network structures, often discovering competitive or novel architectures without explicit human intervention [4, 5, 34, 43]. Within the context of LLM, however, such approaches have been applied almost exclusively to homogeneous Transformer-based architectures, focusing on variations in depth, attention patterns, or parameter allocation. While promising, this line of work remains constrained to a single architectural paradigm.

More recently, alternative sequence modeling approaches have gained attention as potential complements or alternatives to Transformers. State Space Model (SSM), and in particular the Mamba architecture, has emerged as efficient se-

quence model with linear-time complexity and improved scalability to long contexts [6, 11, 30]. Unlike Transformers, which rely on global attention over all past tokens, Mamba employs selective state mechanisms to retain relevant historical information while maintaining computational efficiency [11, 12]. These properties suggest that the dominance of Transformers is no longer absolute and that alternative modeling paradigms may offer superior trade-offs between expressiveness and efficiency [6, 11].

Building on these ideas, hybrid architectures that combine attention-based and state-space-based components—such as Transformer–Mamba models (e.g., Jamba, Zamba)—have recently been proposed [10, 24]. These models show that fundamentally different sequence modeling mechanisms can coexist within a single network, potentially leveraging the strengths of each. However, current hybrid architectures are still manually designed, relying on expert-driven decisions regarding how and where different components should be integrated. As a result, large portions of the hybrid architectural design space remain unexplored, and it is unclear whether existing designs represent optimal or robust solutions.

Crucially, despite the growing diversity of architectural building blocks, EC has not yet been systematically applied to heterogeneous architecture spaces that include both attention-based and state-space-based components. While ENAS has shown promise in automating the design of Transformer-only models, the potential of EC to navigate spaces that mix fundamentally different layer types—such as Transformer and Mamba—remains entirely uninvestigated. Existing work on architecture search for Mamba-related models is scarce, recent, and typically limited to highly constrained or training-free settings rather than evolutionary [9]. This gap is significant: the same evolutionary processes that have uncovered novel Transformer designs might, when applied to a broader palette of architectural primitives, reveal hybrid configurations that outperform both pure Transformer and pure Mamba models, or that offer superior trade-offs between accuracy and efficiency.

The AMALIA project provides a concrete backdrop for this investigation. Aiming to develop large-scale language models for European Portuguese and specialized scientific domains, AMALIA represents a real-world scenario in which automated architecture design could accelerate the development of efficient, domain-specific models. While we do not seek to optimize or deploy production-ready AMALIA models, the tasks relevant to AMALIA—such as text classification and domain adaptation—offer practical evaluation settings that ground our architectural exploration in realistic requirements.

Against this background, we set out to explore whether EC can effectively search the design space of hybrid Transformer–Mamba architectures. Starting from a base implementation of the Mini-Jamba model, we conduct two separate evolutionary experiments, each targeting a different text classification task. The first experiment uses the AG News dataset (English news categorization), while the second uses the PROPOR corpus (Portuguese scientific field classification). For each task, we evolve populations of variable-length genotypes that define the number and type of layers, and perform several independent evolutionary runs. The best genotype from each run then undergoes a final intensive training. We

compare the performance of the evolved architectures against relevant baselines, including the original Mini-Jamba (with pre-trained weights, fine-tuned), as well as Transformer models of comparable size (DistilBERT). Additionally, we carry out a small comparison between an evolved model and a fine-tuned version of the AMALIA model on the Portuguese scientific text classification task, to contextualize the performance achieved under stronger domain requirements.

The main objective of this dissertation is to investigate the feasibility and effectiveness of EC as a search mechanism for hybrid Transformer–Mamba LLM architectures. Rather than proposing a single fixed model, we focus on assessing whether evolutionary methods can systematically explore heterogeneous design spaces and identify competitive or efficient architectures under realistic training constraints. To structure this investigation, we formulate the following research questions:

- **RQ1 — Can we use EC to evolve heterogeneous Transformer-Mamba architectures?** We explore the automation of LLM architectural design by employing an evolutionary algorithm to search for the optimal arrangement of Transformer layers and Mamba layers. We aim to study the viability of finding non-intuitive hybrid topologies that combine the high-quality retrieval capabilities of Transformer layers (based on self-attention) with the linear-time inference efficiency of Mamba (based on state space models). To this end, we devise a representation based on variable-length genotypes and corresponding variation operators.
- **RQ2 — Can evolutionary search discover hybrid architectures with competitive performance compared to manually designed baselines?** We evaluate the evolved architectures on downstream classification tasks and compare their performance against fixed Transformer-only, Mamba-only, and manually specified hybrid models.

1.1 Document Structure

The remainder of this document is organised as follows.

Chapter 2 provides the necessary background and theoretical foundations for this dissertation. It introduces Transformer architectures, State Space Models and Mamba, hybrid sequence modeling approaches, large language models, and the principles of Evolutionary Neural Architecture Search. The chapter concludes with a review of the state-of-the-art and a discussion of the research gaps that motivate this dissertation.

Chapter 3 formalises the problem addressed in this dissertation. It states the core research gap, defines the concrete objectives, and explicitly lists the scope boundaries, constraints, and research questions that guide the experimental investigation.

Chapter 4 describes the methodology adopted for the evolutionary search. It

details the genotype representation, the genetic operators, the fitness evaluation protocol, and the overall evolutionary loop. The chapter also presents the post-evolution intensive training procedure and the datasets used.

Chapter 5 presents the design of the software system that implements the methodology. It covers the overall architecture, the genotype-to-model decoding, the training pipeline, and the data management strategies, emphasising modularity and reproducibility.

Chapter 6 provides the concrete implementation details, including the technology stack, the project directory structure, the main scripts, the hyper-parameters, and the measures taken to ensure reproducibility and efficient memory management.

Chapter 7 reports the experimental results. It first analyses the evolutionary dynamics (fitness progression, architectural convergence, and fitness signal quality) and then presents the final performance of the evolved architectures on the AG News and PROPOR tasks, comparing them against the manually designed baselines.

Chapter 8 discusses the implications of the results in the context of the research questions, identifies the limitations of the study, examines threats to validity, and highlights unexpected findings.

Chapter 9 concludes the dissertation by summarising its main contributions, revisiting the research questions, and outlining directions for future work.

Finally, the appendices provide the complete per-seed results for both tasks and the evolution plots for the PROPOR experiment.

Chapter 2

Background

This chapter provides the conceptual and technical foundations necessary to contextualize the research questions addressed in this dissertation. It introduces the principal sequence modeling paradigms underlying modern language models, alongside the evolutionary optimization methods that motivate the proposed approach.

We begin by reviewing Transformer architectures, which form the backbone of most contemporary large language models, and subsequently discuss state space models and recent alternatives designed to address the computational limitations of attention-based mechanisms. Building on these perspectives, hybrid sequence modeling architectures that combine multiple modeling paradigms are introduced. The chapter then outlines the principles of ENAS, establishing the methodological basis for automated architectural exploration. Finally, the state-of-the-art is reviewed to identify existing limitations and research gaps that motivate the evolutionary investigation of hybrid sequence modeling architectures pursued in this dissertation.

2.1 Transformers

Transformers constitute the dominant architectural paradigm in modern natural language processing and large-scale language modeling. Introduced to address the limitations of recurrent architectures—namely inherently sequential computation, limited parallelism, and difficulty in modeling long-range dependencies due to vanishing and exploding gradients—Transformers replace sequential recurrence with a fully parallelizable mechanism based on self-attention [38]. This design enables efficient modeling of long-range dependencies while significantly improving training scalability on modern hardware.

At the core of the Transformer architecture is the self-attention mechanism, which allows each token in a sequence to attend to all other tokens simultaneously [38]. By computing attention weights based on pairwise token interactions, Transformers can dynamically weight contextual information across an entire sequence. This capability is particularly effective for capturing global dependencies and se-

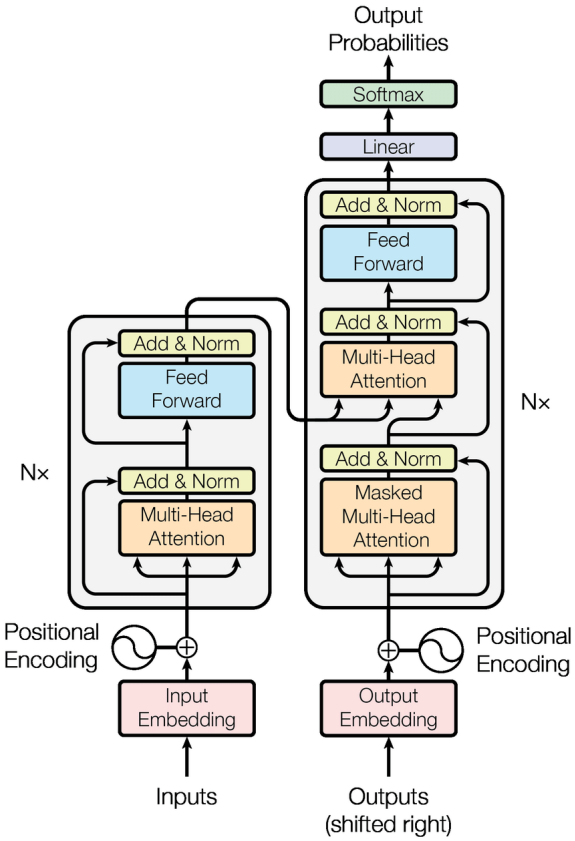

Figure 2.1: Transformer architecture introduced by Vaswani et al.

mantic relationships that may span long textual contexts.

A standard Transformer layer typically consists of a multi-head self-attention module followed by a position-wise feedforward network, with residual connections and layer normalization applied throughout [38]. Stacking multiple such layers yields deep models capable of learning hierarchical representations of language. Positional information, which is not inherently encoded by attention mechanisms, is incorporated through positional embeddings or encodings.

While the original Transformer architecture was proposed with separate encoder and decoder components (Figure 2.1), most contemporary large language models adopt simplified variants tailored to specific tasks. Encoder-only Transformers are particularly prevalent in representation learning and classification tasks, where bidirectional context is essential [19]. In contrast, generative large language models typically rely on decoder-only autoregressive architectures.

Decoder-only Transformers (Figure 2.2) generate text by predicting the next token conditioned on all previous tokens, using causal masking to preserve the autoregressive property. Notably, the base language model of the AMALIA project, EuroLLM [28], follows this decoder-only Transformer design, consistent with its focus on multilingual text generation.

Despite their success, Transformers exhibit several well-known limitations. The computational and memory complexity of self-attention scales quadratically with sequence length, making long-context processing expensive [2, 38]. Furthermore,

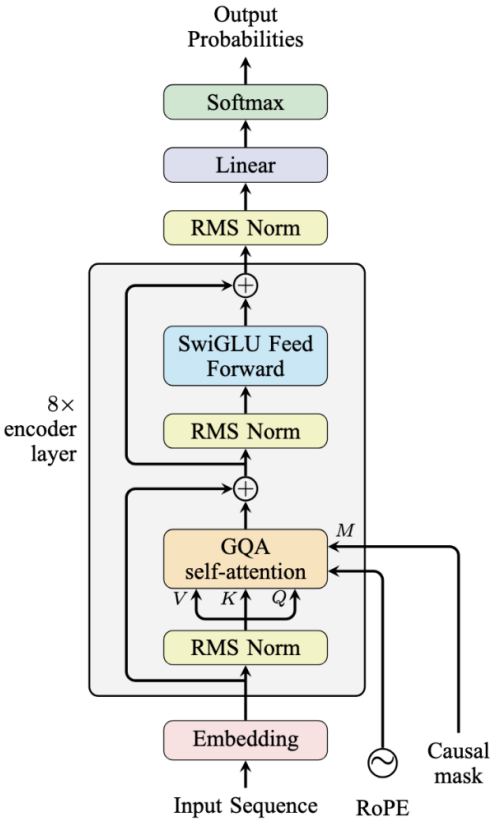

Figure 2.2: Decoder-only Transformer architecture commonly used in large language models.

the lack of an explicit inductive bias for sequential or temporal structure means that Transformers rely heavily on data and scale to learn such patterns implicitly. These limitations have motivated the exploration of alternative sequence modeling paradigms, including State Space Models, discussed in the following section.

2.2 State Space Models and Mamba

SSM represents a fundamentally different approach to sequence modeling, grounded in dynamical systems theory [6, 30]. Unlike Transformers, which rely on pairwise token interactions, SSM models sequences through latent state evolution governed by continuous or discrete-time dynamics. At each time step, the model updates an internal state based on the current input and previous state, producing an output that reflects accumulated historical information [30].

Figure 2.3 illustrates the canonical formulation of a state space model. The input sequence x_t is first projected into the state space through an input matrix B, after which the latent state h_t evolves according to a linear transition defined by matrix A. The output y_t is then generated by projecting the state through matrix C, optionally combined with a direct input-to-output connection via matrix D. This formulation highlights the core idea of SSM: sequence modeling is achieved by repeatedly updating a hidden state that serves as a structured memory of past

inputs.

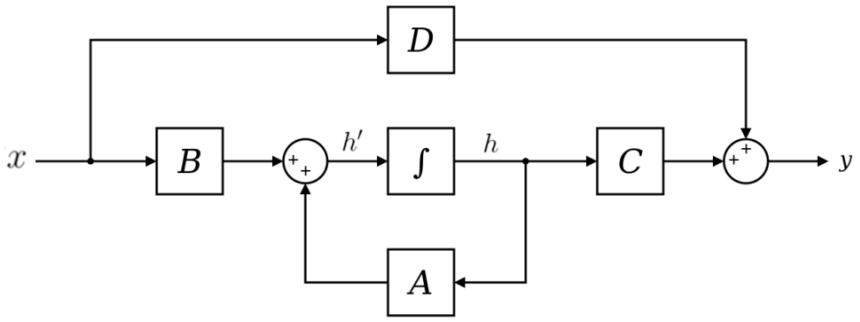

Figure 2.3: Canonical SSM formulation, illustrating latent state evolution governed by input (B), transition (A), and output (C) matrices.

Historically, Recurrent Neural Networks (RNNs) can be viewed as a class of learned discrete-time state space models [6]. While RNNs introduced an explicit inductive bias for sequential data, they suffered from severe training challenges, including vanishing and exploding gradients, limited parallelism, and difficulty modeling long-range dependencies. Subsequent variants such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) alleviated some of these issues but remained constrained by inherently sequential computation [30].

Modern SSM-based architectures aim to retain the favorable inductive bias of recurrence while overcoming the computational limitations of traditional RNNs [6, 11]. Mamba is a prominent example of this new generation of SSM [11]. Its architecture, illustrated in Figure 2.4, builds upon the classical SSM formulation by introducing a *selective state space mechanism*, in which the parameters governing state updates are dynamically conditioned on the input sequence.

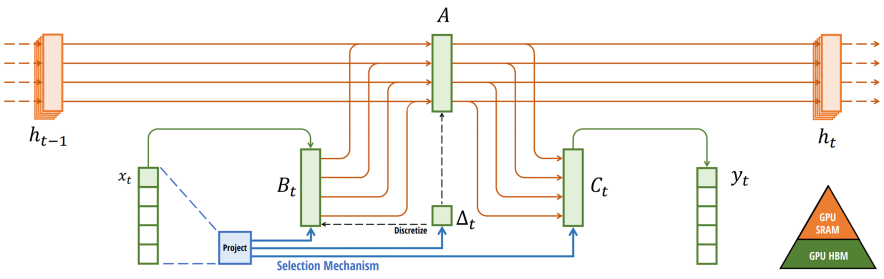

Figure 2.4: Overview of the Mamba selective state space model. Input-dependent projections control the discretization and state update dynamics, enabling selective information flow and hardware-efficient computation.

Unlike the fixed transition dynamics shown in Figure 2.3, Mamba allows the effective state transition and input matrices to vary as a function of the input. This selectivity enables the model to emphasize relevant parts of the sequence while suppressing irrelevant information, addressing a key limitation of earlier linear SSMs [11]. Crucially, this mechanism preserves linear computational complexity with respect to sequence length, making Mamba well suited for long-context modeling [11, 12].

A key innovation in Mamba is its hardware-aware design, which reformulates state updates to be efficiently computed on modern accelerators [11]. By leveraging structured parameterizations, fused operations, and parallelizable scan algorithms, Mamba achieves high throughput while minimizing memory movement between GPU memory hierarchies. As highlighted in Figure 2.4, this design contrasts sharply with attention-based models, whose quadratic memory access patterns become a bottleneck for long sequences [6].

Mamba-2 extends the original architecture by introducing further optimizations and refinements to the selective state space formulation [15, 30]. These enhancements improve numerical stability, expressivity, and training efficiency, addressing limitations observed in earlier SSM designs. Mamba-2 also places greater emphasis on architectural modularity, enabling smoother integration with other neural components and facilitating hybrid architectures that combine state space dynamics with attention-based modules [15, 47].

Overall, Mamba represents a shift toward sequence models with stronger inductive biases for temporal structure and superior scaling properties for long-context processing [11, 30]. In contrast to Transformers, which excel at global contextual reasoning through attention, Mamba emphasizes efficient, structured memory and sequential dynamics [6]. These complementary strengths motivate the investigation of hybrid architectures that combine attention and state space models, forming the basis for the evolutionary approaches explored in this dissertation.

2.3 Hybrid Sequence Modeling Architectures

Hybrid sequence modeling architectures aim to combine the complementary strengths of attention-based Transformer and State Space Model (SSM). While Transformers excel at capturing global contextual relationships through self-attention, they suffer from quadratic computational complexity with respect to sequence length [38]. In contrast, SSM-based models such as Mamba offer linear-time sequence processing and strong inductive biases for temporal structure, but may be less effective at modeling complex global interactions [6, 11]. Hybrid architectures seek to reconcile these trade-offs by integrating attention mechanisms and state space dynamics within a unified model.

Recent work has demonstrated that such integration can yield models that are both computationally efficient and highly expressive. Jamba is a prominent example of this design philosophy, combining Transformer attention blocks with Mamba-style selective state space layers within a single architecture [24]. Rather

than relying on a uniform stack of identical layers, Jamba interleaves different types of processing blocks to balance global reasoning and efficient long-range memory.

Figure 2.5 illustrates the high-level block structure employed by Jamba. The architecture consists of a repeated chain of heterogeneous layers, including standard Transformer layers, Mamba layers, and Mixture of Experts (MoE) variants of both. Attention-based layers are used sparingly to provide global context aggregation, while Mamba layers dominate the depth of the network, enabling efficient linear-time sequence processing. The inclusion of MoE layers further increases model capacity without proportionally increasing computational cost.

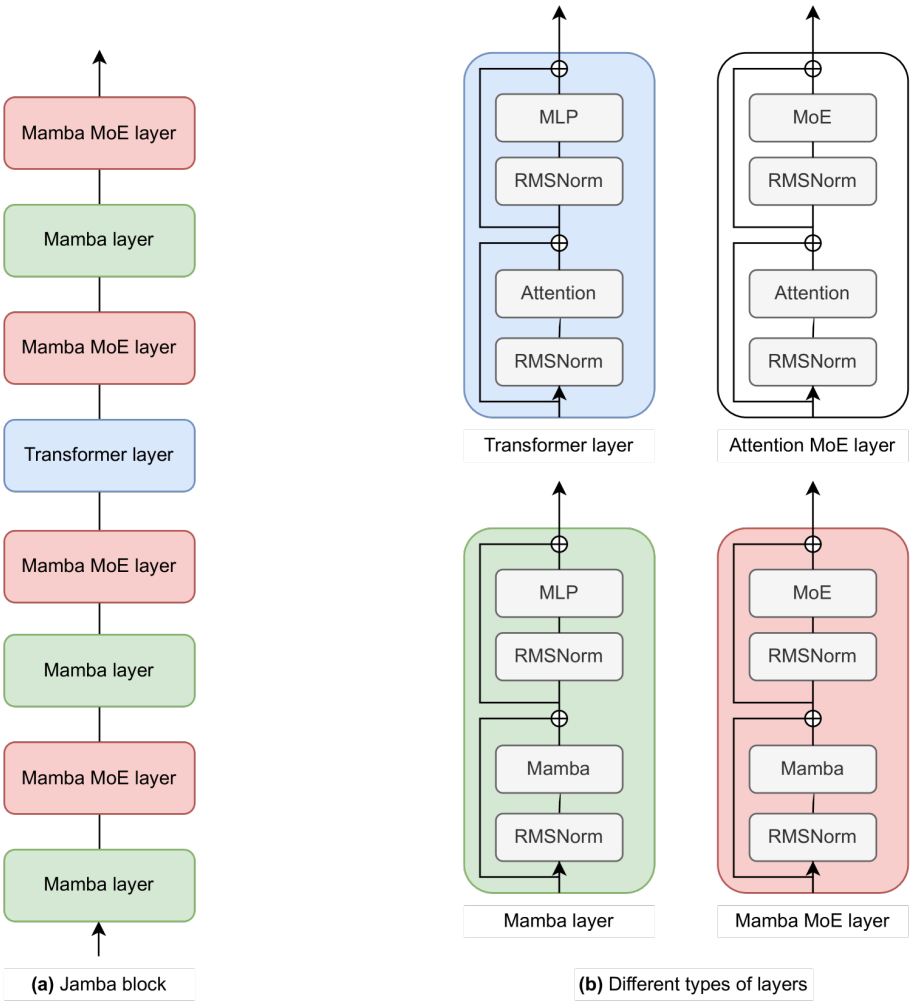

Figure 2.5: High-level block composition of the Jamba architecture. The model interleaves Transformer layers, Mamba layers, and their MoE variants, forming a heterogeneous chain of processing blocks.

This design highlights a key characteristic of modern hybrid architectures: architectural heterogeneity is introduced primarily through the *ordering and frequency* of different block types, rather than through changes to their internal structure. Decisions such as where to place attention layers, how many SSM layers to stack between them, and when to introduce MoE capacity are fixed manually based on expert intuition and empirical validation.

Similarly, Zamba proposes a compact hybrid architecture that blends SSM and attention modules to achieve competitive performance at reduced parameter and compute budgets [10]. These models illustrate that hybrid designs can outperform purely attention-based or purely SSM-based architectures under practical resource constraints, particularly for long-context tasks. Importantly, both Jamba and Zamba rely on manually designed architectural patterns, reflecting expert assumptions about how different sequence modeling primitives should be composed.

Beyond language modeling, hybridization has also been explored in multi-modal settings. Models such as ML-Mamba and Cobra extend the Mamba framework by incorporating attention-based components to better handle cross-modal interactions, further highlighting the flexibility of hybrid designs [15, 47]. These approaches reinforce the view that attention and state space mechanisms are not competing paradigms, but rather complementary building blocks that can be composed to address diverse modeling requirements.

Despite their promise, existing hybrid architectures remain largely handcrafted, with limited systematic exploration of the design space. Choices such as the ordering of attention and SSM layers, the relative depth allocated to each component, and the granularity of hybridization are typically fixed *a priori*. This results in a vast combinatorial search space that is difficult to explore manually and may contain architectures with superior performance–efficiency trade-offs.

This observation motivates the application of automated architecture search methods to hybrid sequence models. In particular, ENAS offers a natural framework for exploring heterogeneous search spaces composed of multiple architectural primitives. By treating attention and state space modules as evolvable building blocks, evolutionary approaches can systematically discover novel hybrid architectures that balance expressivity, efficiency, and scalability. This perspective underpins the methodology proposed in this dissertation.

2.4 Large Language Models

Large Language Models (LLMs) are deep neural networks trained on massive corpora of text to perform a broad variety of natural language tasks. These models are typically built on Transformer architectures or closely related variants and are trained using large-scale self-supervised objectives, enabling them to capture rich contextual representations and exhibit emergent capabilities beyond traditional task-specific models.

One of the earliest highly influential LLM was BERT, a bidirectional Transformer-based model that significantly advanced state-of-the-art performance across multiple natural language understanding benchmarks by leveraging deep, contextualized representations learned through masked language modeling and next-sentence prediction [7]. BERT's success demonstrated the effectiveness of large-scale pre-training on unlabeled text followed by task-specific fine-tuning, establishing a dominant paradigm in modern Natural Language Processing (NLP).

Beyond masked language models, GPT-3 introduced a new scaling-driven approach to autoregressive language modeling, showing that increasing model size leads to strong few-shot and zero-shot performance without explicit fine-tuning [3]. With 175 billion parameters, GPT-3 demonstrated that general-purpose language capabilities can emerge from large-scale pre-training alone, reshaping research priorities toward scale and architectural efficiency.

More recently, the LLMs landscape has expanded with models such as LLaMA [37] and BLOOM [20], which explore trade-offs between performance, training efficiency, multilingual coverage, and open-access design. These models have reinforced the role of LLMs as foundational components in modern Artificial Intelligence systems while simultaneously highlighting the growing computational, environmental, and engineering costs associated with large-scale model training and deployment.

Despite their impact, the architectures underlying most LLMs remain manually designed, relying on incremental refinements of existing Transformer-based templates. As model scale continues to increase, this manual design process becomes increasingly expensive and restrictive. These challenges motivate the exploration of automated and principled approaches to architectural design, particularly methods capable of discovering efficient and expressive model structures for large-scale language modeling.

2.5 Evolutionary Neural Architecture Search

Neural Architecture Search (NAS) seeks to automate the design of neural network architectures, reducing reliance on manual, expert-driven model engineering. Within this broader field, ENAS leverages principles from EC to explore complex, high-dimensional architecture spaces in a population-based and stochastic manner. Evolutionary approaches are particularly attractive for NAS due to their flexibility, gradient-free optimization, and ability to handle discrete, heterogeneous, and multi-objective design spaces [5].

Unlike gradient-based NAS methods, which often require differentiable search spaces and strong structural constraints, evolutionary methods can naturally operate over diverse architectural components, variable-length representations, and non-differentiable design choices. This makes ENAS well suited for exploring heterogeneous architectures, such as those combining attention-based and state-space-based modules. As a result, evolutionary NAS has been successfully applied across a wide range of domains, including convolutional networks, vision transformers, and large-scale deep learning systems [26, 34].

This section introduces the fundamental concepts of evolutionary algorithms and discusses how they are adapted for neural architecture search, laying the foundation for their application to hybrid sequence modeling architectures in this dissertation.

2.5.1 Evolutionary Algorithms

Evolutionary algorithms (EAs) are a class of population-based optimization methods inspired by natural evolution. They operate by maintaining a population of candidate solutions, which evolve over successive generations through the application of variation operators such as mutation and crossover, guided by a fitness-based selection process [14, 35]. Over time, this iterative process biases the population toward increasingly fit solutions.

Figure 2.6 illustrates the canonical evolutionary algorithm cycle. The process begins with the initialization of a population of candidate solutions, which are evaluated according to a fitness function. Based on these evaluations, a selection mechanism chooses parent individuals that are then modified by variation operators to produce offspring. A replacement strategy determines how offspring are incorporated into the population, forming a new generation. This cycle is repeated until a termination criterion is met, with the fittest individuals emerging over successive generations.

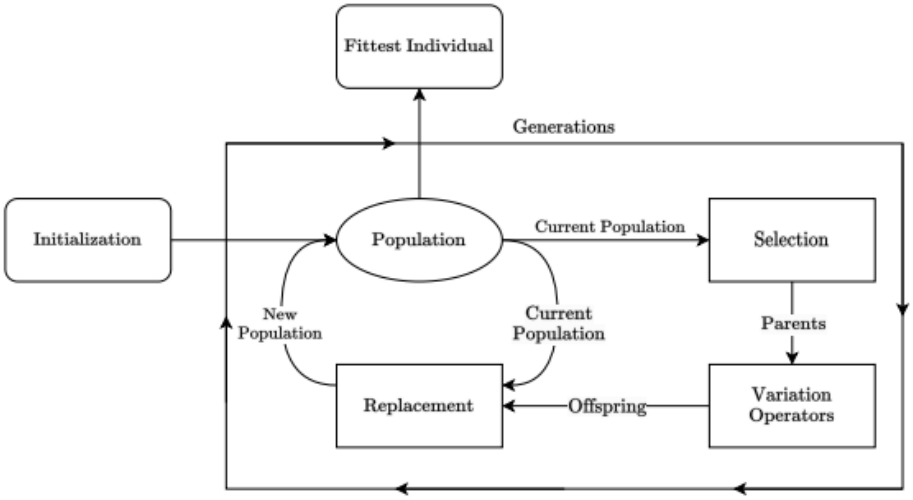

Figure 2.6: Evolutionary Algorithm Cycle.

A defining characteristic of evolutionary algorithms is their reliance on fitness evaluations rather than gradient information. This property allows EAs to optimize objectives that are non-differentiable, noisy, multi-modal, or computationally expensive. As a result, evolutionary methods have been widely adopted in settings where traditional optimization techniques struggle, including reinforcement learning, control, and neural architecture optimization [32, 41].

Several families of evolutionary algorithms are particularly relevant in the context of neural architecture search. Genetic Algorithms (GAs) typically represent candidate solutions as discrete genomes and apply crossover and mutation operators to explore the search space. Evolution Strategies (ES), in contrast, focus on optimizing real-valued parameters through stochastic perturbations and population-level statistics, offering strong scalability properties [32]. Neuroevolution approaches, such as NEAT, explicitly evolve both the topology and parameters of neural networks, enabling the discovery of novel architectures without

predefined structural constraints [35].

Beyond single-objective optimization, evolutionary algorithms naturally extend to multi-objective settings, where trade-offs between competing goals—such as accuracy, computational cost, memory footprint, and latency—must be considered simultaneously. Population-based methods can approximate Pareto fronts of optimal solutions, making them especially well suited for architecture search under real-world constraints [16, 26]. These properties form the methodological basis for ENAS approaches discussed in the following subsection.

2.5.2 Evolutionary Algorithms Applied to Neural Architecture Search

ENAS applies evolutionary algorithms to the automated discovery of neural network structures, encoding architectural decisions into evolutionary representations and optimizing them through population-based search. Compared to gradient-based NAS methods, evolutionary approaches offer greater flexibility in defining search spaces, supporting discrete, variable-length, and heterogeneous architectural representations without requiring differentiability [5].

Genetic Algorithms for NAS. GAs are among the earliest and most widely used evolutionary techniques for architecture search. In GA-based NAS, candidate architectures are typically encoded as genomes describing layer types, connectivity patterns, or hyper-parameters, which are evolved through mutation and crossover operators [35]. In such GAs, variation operators typically include crossover (e.g., one-point or uniform crossover of architectural components) and mutation (e.g., adding/removing layers, changing hyper-parameters, or perturbing connectivity). Early work demonstrated that evolutionary methods could discover competitive neural architectures without manual design, a paradigm later scaled to deep convolutional networks and large search spaces [31].

In modern NAS settings, GAs have been applied to model compression, pruning, and efficiency-aware optimization. Methods such as EvoPress and EAPruning evolve architectures under resource constraints, explicitly balancing predictive performance against model size, latency, or energy consumption [23, 33]. These approaches highlight the suitability of genetic search for multi-objective architecture optimization.

Evolution Strategies and Scalable NAS. ES represent an alternative evolutionary paradigm that emphasizes scalability and parallelism. Rather than relying on discrete crossover operations, ES methods optimize architectures through stochastic perturbations and population-level statistics [32]. ES-based NAS has been shown to scale effectively to large models and distributed computing environments, making it attractive for architecture search in high-dimensional spaces [41].

Population Based Training (PBT) further extends this idea by jointly optimizing

model parameters and hyper-parameters during training, periodically exchanging information between population members [16]. Multi-objective variants of PBT enable the exploration of trade-offs between accuracy and efficiency, which is particularly relevant for large-scale neural architectures [21].

Multi-population and Structured Evolutionary Search. To improve exploration and avoid premature convergence, several ENAS methods employ multiple interacting populations or structured evolutionary dynamics. Multi-population NAS frameworks alternate between exploration-focused and exploitation-focused populations, enabling more efficient coverage of large and rugged search spaces [45]. Similarly, approaches such as EG-NAS and EG-ENAS introduce fast evolutionary exploration strategies that decouple architectural mutation from expensive full training [42, 48].

More recently, hybrid evolutionary frameworks have been proposed that integrate evolutionary search with generative or training-free evaluation mechanisms. For example, Evolution Meets Diffusion explores neural architecture generation through diffusion-based priors guided by evolutionary objectives, further expanding the scope of ENAS methodologies [25].

Applications to Sequence Models and Transformers. Evolutionary NAS has increasingly been applied to sequence models, including Transformers and large language models. The Evolved Transformer demonstrated that evolutionary search can discover competitive attention-based architectures without manual design [34]. Subsequent work has explored evolutionary pruning, architectural adaptation, and efficiency optimization for Transformer-based models in both vision and language domains [9, 26].

Despite these advances, most existing ENAS approaches for sequence modeling remain constrained to homogeneous Transformer architectures. Only limited work has explored evolutionary optimization in heterogeneous or hybrid architecture spaces, particularly those combining attention mechanisms with alternative sequence modeling paradigms. This limitation motivates the exploration of evolutionary methods for hybrid Transformer-Mamba architectures pursued in this dissertation.

2.6 State of the Art

This section reviews prior work at the intersection of EC and neural architecture design for sequence models. It focuses on ENAS for Transformer architectures, evolutionary optimization techniques applied to large language models, and emerging efforts that move beyond Transformer-only design spaces. By situating recent advances within these themes, this section identifies key limitations of existing approaches and motivates the need for evolutionary methods capable of exploring hybrid and heterogeneous sequence modeling architectures.

2.6.1 Evolutionary NAS for Transformer Architectures

ENAS has been extensively explored in the context of Transformer-based models, primarily with the goal of improving performance, efficiency, and architectural adaptability while reducing reliance on manual design. Early work demonstrated that evolutionary methods are capable of discovering competitive attention-based architectures without relying on fixed Transformer templates.

A seminal contribution in this area is the *Evolved Transformer*, which applied evolutionary search to the design of encoder–decoder attention architectures for sequence-to-sequence tasks [34]. The approach used tournament selection with an aging mechanism and a set of mutation-only variation operators (without crossover) that directly manipulated the cell structure: operators included inserting new layers, removing existing layers, replacing a layer with a different type (e.g., attention vs. convolution), and modifying layer-internal hyper-parameters such as the number of attention heads or the activation function. By evolving the structure of attention blocks, feedforward components, and connectivity patterns, the method identified architectures that matched or exceeded the performance of hand-designed Transformers while using fewer parameters. This work provided early evidence that evolutionary algorithms can effectively navigate the complex architectural design space of attention-based models.

Subsequent research expanded evolutionary NAS to explore a broader range of Transformer design dimensions, including depth, width, attention head configurations, and block-level composition. Several approaches focused on multi-objective optimization, balancing predictive performance against computational cost, memory footprint, or latency constraints [26, 49]. These methods demonstrated that evolutionary search can identify Pareto-optimal Transformer variants that achieve favorable trade-offs between accuracy and efficiency.

Evolutionary techniques have also been applied to structural pruning and compression of Transformer architectures. Methods such as evolutionary pruning and dynamic compression evolve layer-level or block-level removal strategies, enabling substantial reductions in model size while maintaining performance [23, 33]. Related work has explored evolutionary adaptation of Transformer architectures under resource constraints, particularly in vision and multimodal settings [22].

Beyond static architecture design, evolutionary approaches have been combined with population-based training and evolution strategies to optimize Transformer training dynamics and hyper parameters [16, 27]. These methods emphasize adaptability and robustness during training rather than fixed architectural discovery, highlighting the versatility of EC in Transformer optimization.

Comprehensive surveys of Transformer NAS methods indicate that most existing evolutionary approaches operate within largely homogeneous Transformer search spaces, focusing on variations of attention-based blocks and feedforward components [40, 44]. While effective, this focus inherently limits the exploration of fundamentally different sequence modeling mechanisms.

Overall, the state-of-the-art demonstrates that evolutionary NAS is a powerful

tool for optimizing Transformer architectures, enabling automated discovery of efficient and high-performing attention-based models. However, the vast majority of existing work remains confined to pure Transformer design spaces, leaving heterogeneous and hybrid architectures—particularly those combining attention with alternative sequence modeling paradigms—largely unexplored. This gap motivates the investigation of evolutionary search methods beyond standard Transformer architectures, as pursued in this dissertation.

2.6.2 Evolutionary Optimization of Large Language Models

As large language models scale to billions of parameters, direct evolutionary search over full architectures becomes computationally prohibitive. Consequently, recent research has shifted toward applying EC techniques to *optimize, adapt, and combine* pre-trained LLMs rather than evolving them from scratch. In this setting, EC is primarily employed at higher levels of abstraction, such as model composition, weight adaptation, hyperparameter evolution, and training dynamics.

One prominent line of work explores evolutionary optimization for *model merging and weight-space manipulation*. Akiba et al. introduced evolutionary optimization of model merging recipes, where populations of candidate merge configurations are evolved to combine multiple pre-trained models into a single performant model without additional large-scale training [1]. This approach demonstrates that evolutionary search can effectively navigate complex weight-space interactions, yielding merged LLMs that outperform manually designed merging strategies.

Closely related to this idea, Du et al. proposed evolving weight combinations for knowledge fusion across language models [8]. Rather than designing fusion mechanisms heuristically, evolutionary algorithms are used to search for optimal weightings that integrate complementary knowledge from multiple models. These methods highlight the suitability of EC for optimization problems where gradients are unavailable or unreliable, and where evaluation can be performed at the level of downstream task performance.

Beyond weight-level optimization, evolutionary methods have been investigated for *structural adaptation and pruning* of large language models. Structural NAS techniques have been applied to identify redundant layers or components within LLMs, enabling parameter reduction while preserving performance [18]. Although constrained to relatively coarse-grained architectural modifications, these approaches demonstrate that evolutionary search remains viable even in large-scale model regimes when the search space is carefully restricted.

More broadly, recent surveys emphasize that EC offers complementary advantages to gradient-based optimization when applied to LLMs, particularly in settings involving discrete design choices, multi-objective trade-offs, or black-box evaluation pipelines [4, 39]. Evolutionary strategies have been proposed for optimizing prompts, training curricula, hyper-parameters, and even interaction policies between LLMs and external systems, extending the applicability of EC well beyond architecture search alone.

Despite these advances, most evolutionary optimization techniques for LLMs operate *around* fixed architectures rather than within the architectural design space itself. The dominant focus remains on Transformers, and only limited work explores evolutionary optimization in alternative or hybrid sequence modeling paradigms. As a result, the application of evolutionary methods to heterogeneous architectures—particularly those integrating attention-based models with state space dynamics—remains an open and underexplored research direction.

In summary, EC has emerged as a practical and flexible tool for optimizing large language models at scale, enabling effective exploration of weight-space, structural, and training-level design choices. However, its potential for guiding architectural innovation beyond standard Transformer designs has yet to be fully realized, motivating the investigation of evolutionary optimization in hybrid and non-attention-based LLM architectures pursued in this dissertation.

2.6.3 Evolutionary Search Beyond Transformers

While the Transformer architecture has dominated sequence modeling research for several years, its limitations in terms of computational efficiency, memory usage, and scalability have motivated the exploration of alternative sequence modeling paradigms. In response, recent work has begun to investigate architectures that move beyond pure self-attention, including state space models (SSMs), recurrent hybrids, convolutional sequence models, and other attention-free or partially attention-based designs. However, ENAS in these broader architectural spaces remains comparatively underexplored.

Early evolutionary NAS frameworks were largely developed for convolutional and recurrent neural networks, focusing on homogeneous architecture families with well-understood inductive biases. As a result, most evolutionary methods for sequence modeling have inherited a strong architectural bias toward Transformer-style designs, often restricting the search space to variations in attention heads, feed-forward widths, or layer depth. While effective for incremental improvements, such constraints limit the discovery of fundamentally different sequence modeling strategies.

Recent advances in state space models, such as S4 and Mamba, have demonstrated that competitive sequence modeling performance can be achieved without explicit attention mechanisms, offering linear-time complexity and improved efficiency on long sequences. Despite their growing adoption in hand-designed architectures, these models have seen limited integration into evolutionary NAS pipelines. Existing evolutionary approaches rarely consider heterogeneous search spaces that allow the combination of attention-based and state-space-based components within a single architecture.

A small number of works have begun to move in this direction by incorporating non-attention modules into NAS search spaces or by exploring hybrid architectures through manual design. However, these efforts typically rely on gradient-based NAS or heuristic design choices rather than population-based evolutionary search. Consequently, the potential of EC to navigate complex, discrete, and

heterogeneous architecture spaces—where gradient signals are weak or unavailable—remains largely untapped.

This gap highlights a key opportunity for evolutionary NAS: enabling the automated discovery of hybrid sequence modeling architectures that combine complementary modeling paradigms. By extending evolutionary search beyond Transformer only spaces, it becomes possible to explore architectural trade-offs between expressivity, efficiency, and scalability in a principled and systematic manner. Addressing this challenge is particularly relevant for modern large language models, where architectural efficiency and modularity are increasingly important constraints.

2.6.4 Summary of Limitations and Research Gaps

The literature reviewed in the preceding sections demonstrates substantial progress in both EC for neural architecture search and the development of efficient alternatives to Transformer-based sequence modeling. However, when these two research directions are considered jointly, several limitations emerge that motivate further investigation.

First, evolutionary approaches applied to Transformer models typically operate within narrowly constrained architectural search spaces. Most methods focus on optimizing parameters such as depth, width, attention configurations, or pruning strategies while preserving the core self-attention paradigm. While effective for refining existing architectures, this constraint fundamentally limits the ability of evolutionary search to discover qualitatively different sequence modeling mechanisms.

Second, as language models scale to billions of parameters, EC has increasingly been applied at higher levels of abstraction. Recent work emphasizes model merging, weight adaptation, hyperparameter evolution, and training dynamics, deliberately avoiding direct architectural evolution due to its prohibitive computational cost. Consequently, architectural innovation in large language models has largely remained decoupled from evolutionary search.

Third, recent advances in state space models and hybrid Transformer-SSM architectures demonstrate that attention-free or partially attention-based designs can achieve competitive performance with improved efficiency. Models such as Mamba and hybrid architectures like Jamba highlight the viability of alternative sequence modeling paradigms. However, these architectures are almost exclusively the result of manual design and expert-driven iteration. To date, evolutionary methods have seen limited application in exploring heterogeneous architecture spaces that combine attention-based and state-space-based components.

Taken together, these observations expose a broader research gap: the absence of evolutionary methods capable of systematically exploring hybrid sequence modeling architectures that extend beyond Transformer-only designs while remaining computationally feasible. Addressing this gap in full would require evolutionary optimization over large-scale heterogeneous architecture spaces combined with

extensive pretraining or fine-tuning at LLM scale, an undertaking that realistically spans years of research and substantial computational resources.

In this dissertation, we explicitly acknowledge the scope of this broader challenge while deliberately focusing on a constrained and tractable subset of the problem. Rather than evolving large language models from scratch or conducting unconstrained architecture search, we investigate evolutionary strategies for exploring restricted hybrid architecture spaces that combine Transformer and Mamba components under practical computational budgets. This design choice enables a focused study of the interaction between evolutionary search and hybrid sequence modeling while remaining feasible within the timeframe of a master's dissertation.

Table 2.1 summarizes the key characteristics of the representative sequence modeling approaches discussed in this chapter. The comparison highlights a clear gap in the existing literature: while evolutionary methods have been successfully applied to Transformer architectures, no prior work jointly integrates EC with hybrid Transformer-Mamba sequence modeling architectures.

Table 2.1: Comparison of representative sequence modeling approaches.

Approach	**Uses EC**	**Transformer**	**Mamba**	**Hybrid**
Evolved Transformer [34]	✓	✓	–	–
Mamba [11]	–	–	✓	–
Jamba [24]	–	✓	✓	✓
This dissertation	✓	✓	✓	✓

As illustrated in Table 2.1, existing approaches typically satisfy only a subset of these criteria. Evolutionary methods remain largely confined to Transformer-centric design spaces, while hybrid architectures rely on manual construction. The approach pursued in this dissertation aims to bridge this gap by applying evolutionary search to a constrained hybrid architecture space, balancing architectural expressivity with computational feasibility.

Chapter 3

Problem Definition and Objectives

As discussed in Chapter 2, EC has proven effective for automating the design of Transformer-based architectures, yet it has not been systematically applied to heterogeneous search spaces that combine attention-based and state-space-based components. Recent advances in hybrid architectures such as Jamba and Zamba demonstrate that manually designed combinations of Transformer and Mamba layers can achieve competitive performance with improved efficiency. However, the design of these hybrid architectures remains an expert-driven, manual process, and vast portions of the architectural design space—particularly the optimal arrangement and proportion of different layer types—remain unexplored.

The core problem addressed in this dissertation is therefore the absence of empirical evidence on whether EC can effectively explore such heterogeneous design spaces and discover competitive hybrid Transformer–Mamba architectures under practical training constraints. We specifically focus on the search over two key architectural dimensions: the number of layers and the type of each layer (Transformer or Mamba), while keeping the internal structure of individual layers fixed.

The main objective of this dissertation is to investigate the feasibility and effectiveness of an evolutionary algorithm for exploring the design space of hybrid Transformer–Mamba architectures. To achieve this, we pursue the following concrete goals:

1. Design and implement an evolutionary search framework that operates over variable-length genotypes, where each gene specifies whether a layer is a Transformer block or a Mamba block.
2. Evaluate the evolutionary search on two text classification tasks with different characteristics: the AG News dataset (English, four balanced classes) and the PROPOR scientific field classification corpus (Portuguese, five classes).
3. For each task, perform multiple independent evolutionary runs and subsequently do an intensive training of the best discovered architecture on the full training set.

4. Compare the performance of the evolved architectures against manually designed baselines.

5. Analyze the architectural patterns that emerge from the evolutionary search over successive generations, and assess whether the search converges to consistent structural motifs (e.g., Mamba-only shallow architectures).

To keep the investigation tractable and reproducible within the available computational budget, we impose the following scope boundaries:

- **Architectural dimensions:** The search is limited to the depth (number of layers) and the type of each layer (Transformer vs. Mamba). The internal configuration of each layer—such as the number of attention heads, the state dimension, the expansion factor, and the placement of Mixture-of-Experts (MoE) modules—is fixed and inherited from the Mini-Jamba configuration.
- **Training budget per individual:** During evolution, each candidate architecture is trained for a fixed number of steps on a subset of the training data. This limited budget is a practical necessity that influences the fitness landscape.
- **Evaluation protocol:** Within each generation, all individuals are trained on the same sequence of mini-batches (controlled via a fixed random seed) to ensure fair comparison. The fitness of an individual is its F1-score on a held-out validation set.
- **Computational resources:** All experiments were conducted on a single NVIDIA GeForce RTX 3080 Ti with 12 GiB of VRAM. The available memory imposed constraints on the maximum batch size and sequence length.

The goals and constraints outlined above are captured by the following two research questions, which guide the experimental design and the analysis:

- **RQ1 — Can we use EC to evolve heterogeneous Transformer-Mamba architectures?** This question investigates whether an evolutionary algorithm with variable-length genotypes and appropriate variation operators (bit-flip mutation, structural insertion/deletion) can effectively navigate a search space that mixes two fundamentally different layer types.
- **RQ2 — Can evolutionary search discover hybrid architectures with competitive performance compared to manually designed baselines?** This question assesses the practical value of the evolved architectures by comparing their classification performance and inference latency against fixed Transformer-only, Mamba-only, and hybrid baselines, all under identical training protocols.

Chapter 4

Methodology

This chapter describes the methodology adopted for investigating the use of EC to automatically design hybrid Transformer–Mamba architectures. The approach centers on an evolutionary algorithm that searches a space of variable-length layer sequences, evaluates each candidate on a text classification task under a fixed training budget, and finally compares the best discovered architectures against manually designed baselines.

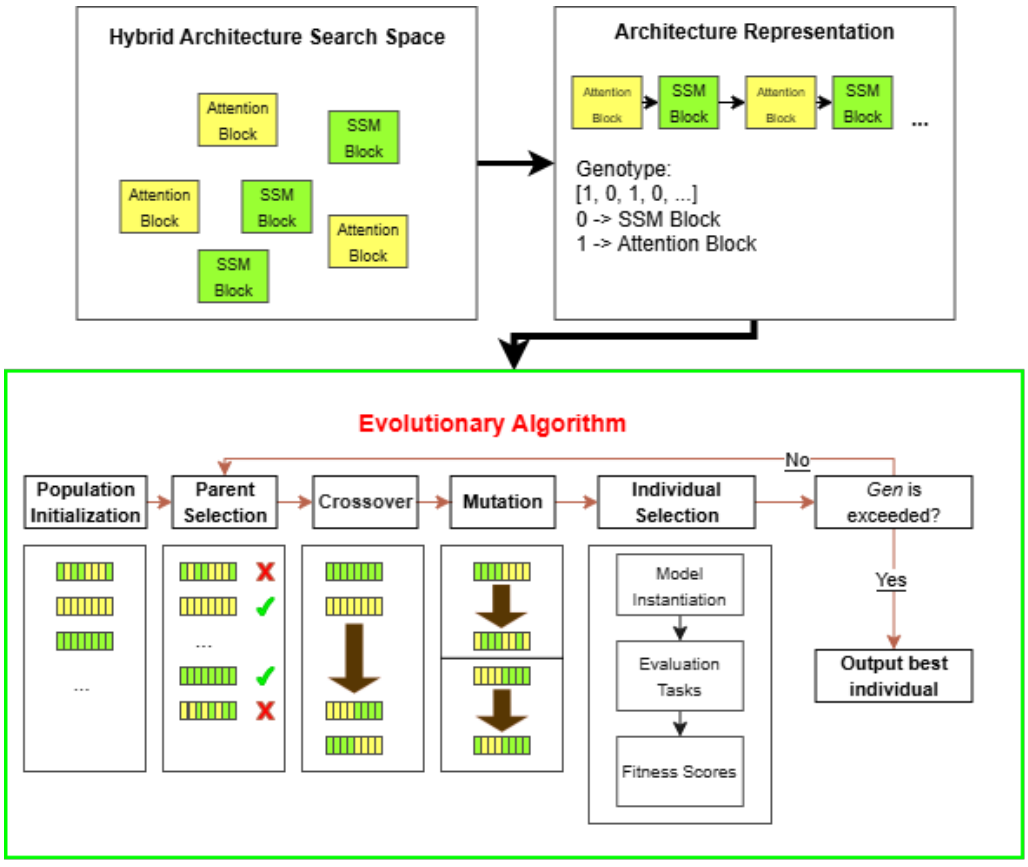

Figure 4.1: High-level overview of the methodology.

The high-level workflow is illustrated in Figure 4.1. An initial population of random architectures is generated and evaluated on a downstream task. The

best-performing individuals are selected to produce offspring through crossover and mutation, and the process repeats for a fixed number of generations. After the evolutionary run, the most promising architecture is further trained on the full dataset to obtain its final performance, which is then compared against hand-crafted baselines.

4.1 Search Space and Evolutionary Algorithm

4.1.1 Genotype Representation

A candidate architecture is represented as a **variable-length binary list**, called a *genotype*. Each gene takes the value 0 (Mamba layer) or 1 (Transformer layer). The order of the genes directly determines the sequence of layers in the model: position i of the genotype specifies the type of the i-th layer. The length of the genotype is bounded between a minimum of 4 and a maximum of 20 layers. Every layer is built using the same internal configuration inherited from the Mini-Jamba model, and only the layer type is subject to evolution.

4.1.2 Genetic Operators

The evolutionary search employs three operators: **crossover**, **bit-flip mutation**, and **structural mutation**.

Crossover. Single-point crossover is applied with probability 0.8. A random cut point is chosen in the shorter of the two parent genotypes, and the prefix of the first parent is concatenated with the suffix of the second parent. If the resulting child exceeds the maximum allowed length, it is truncated; if it is shorter than the minimum length, random genes are appended to reach the minimum.

Bit-flip mutation. Occurs with probability 0.5. When activated, each gene of the genotype is flipped ($0 \leftrightarrow 1$) with probability $1/L$, where L is the current length of the genotype. This means that, on average, one gene is altered per application, providing a modest exploration step that can change a layer type without modifying the depth.

Structural mutation. With a probability of 0.25, the genotype length is modified. The operator is equally likely to insert a new random gene at a uniformly chosen position (if the length is below the maximum) or to delete a randomly selected gene (if the length is above the minimum). When the genotype is at one of the length bounds, only the permitted operation is attempted. This operator allows the search to dynamically adjust the depth of architectures.

Both mutation operators are applied to the offsprings. Because structural changes break the one-to-one mapping between parent and child layers, we do **not** perform weight inheritance: every offspring is initialized with fresh random weights.

4.1.3 Fitness Evaluation

The fitness of an individual is its **F1-score** on a held-out validation set after a short, fixed-budget training. The protocol is identical for every individual:

- A Jamba-style model is built according to the genotype (using the Mini-Jamba configuration) with random weights. A linear classification head is added on top of mean-pooled hidden states.
- The model is trained for a fixed number of optimizer steps on a subset of the training data, using the AdamW optimizer with learning rate 4×10^{-4}.
- After training, the model is evaluated on the validation set, and the F1-score is recorded.

To guarantee a fair comparison within each generation, the random seed used by the data loader is set to a generation-specific value, so that every individual sees exactly the same sequence of mini-batches.

4.1.4 Evolutionary Loop

The algorithm follows a generational cycle with elitism (population size 30, 100 generations):

1. **Initialization.** Genotypes are created with random lengths between 4 and 20 layers and random gene values. For the 300-step AG News runs, we additionally tested a variant where the initial population was forced to have at least 10 layers, to examine whether a deeper starting point would bias the search. This variant produced the same final architectures and its results are included in the main analysis.
2. **Evaluation.** All individuals are trained and evaluated as described above.
3. **Selection and reproduction.** The population is sorted by fitness. The best individual is kept unchanged (elitism). For each remaining slot, two parents are chosen via tournament selection (size 3) and produce one child through crossover and mutation.
4. **Iteration.** Steps 2–3 are repeated for 100 generations. After each generation, fitness statistics and architectural metrics (mean depth, proportion of Mamba vs. Transformer layers) are logged.

4.2 Post-Evolution Training and Comparison

After each independent evolutionary run, the genotype with the highest fitness across all generations is selected. A fresh model is built with that genotype, and its weights are initialized from the checkpoint saved during evolution. The model is then further trained on the **full** training set of the respective task for several epochs (4 epochs for AG News, 8 epochs for PROPOR) using a lower learning rate (3×10^{-5}) and a linear warm-up schedule. This intensive training phase allows the architecture to reach its full potential, and the resulting model serves as the final representative of that evolutionary run.

For the **AG News** task, the evolved architectures (one per seed) are compared against four baselines:

- **Mini-Jamba (original).** The 16-layer alternating Mamba–Transformer architecture, fine-tuned from its pre-trained weights [24, 36].
- **DistilBERT base (66 M parameters).** A distilled Transformer, fine-tuned on AG News [17].
- **BERT base (110 M parameters).** The BERT model fine-tuned on AG News [13].
- **Jamba-style deep hybrid (32 layers).** A deeper architecture following the original Jamba sparse-attention pattern (attention layers at indices 4, 12, 20, 28) [24], trained from scratch under the same protocol, to assess the value of evolutionary depth discovery.

For the **PROPOR** task, the comparison is made against a single baseline:

- **AMALIA (fine-tuned).** A version of the AMALIA model (a large language model for European Portuguese) fine-tuned on the PROPOR scientific field classification task, serving as the domain-specific reference.

All comparisons are reported in terms of F1-score, accuracy, precision and recall. The evolutionary algorithm itself uses only the F1-score on the validation set as the fitness signal.

4.3 Tasks and Datasets

The methodology is evaluated on two text classification tasks that differ in language, domain, and difficulty.

AG News (English). AG News is a news categorization dataset comprising 120,000 training samples and 7,600 test samples, evenly distributed across four classes: *World*, *Sports*, *Business*, and *Sci/Tech*. Each sample consists of a short news

headline and a brief description. The dataset, introduced by Zhang et al. [46], is widely used as a benchmark for text classification, particularly for evaluating lightweight or efficiency-oriented architectures. Following common practice, we truncate all texts to 128 tokens, which is sufficient to cover the majority of the content in this dataset.

PROPOR FOS Classification (Portuguese). The PROPOR FOS Classification dataset is a collection of Portuguese academic works (theses and dissertations) labeled according to the OECD Fields of Science (FOS) classification [29]. The task is a 5-class classification problem: given the **title**, **keywords**, and **abstract** of a thesis, the model must predict its scientific field. We use the official dataset splits: the training set (7,756 samples) for evolution and fine-tuning, and the dedicated test set for final evaluation. Because the abstracts can be substantially longer than the texts in AG News, we truncate sequences to 512 tokens, which captures the most informative portion of the combined title, keywords, and abstract text while remaining compatible with our GPU memory constraints.

Chapter 5

System or Model Design

This chapter details the architecture and design of the software system that implements the evolutionary search and the intensive training of hybrid Transformer–Mamba architectures. While Chapter 4 described the methodological choices, here we focus on how the algorithms, models, and data pipelines are realized in a modular and reproducible codebase.

5.1 Overall System Architecture and Genotype Decoding

The system is organized into two main phases that share a common model library (Figure 5.1).

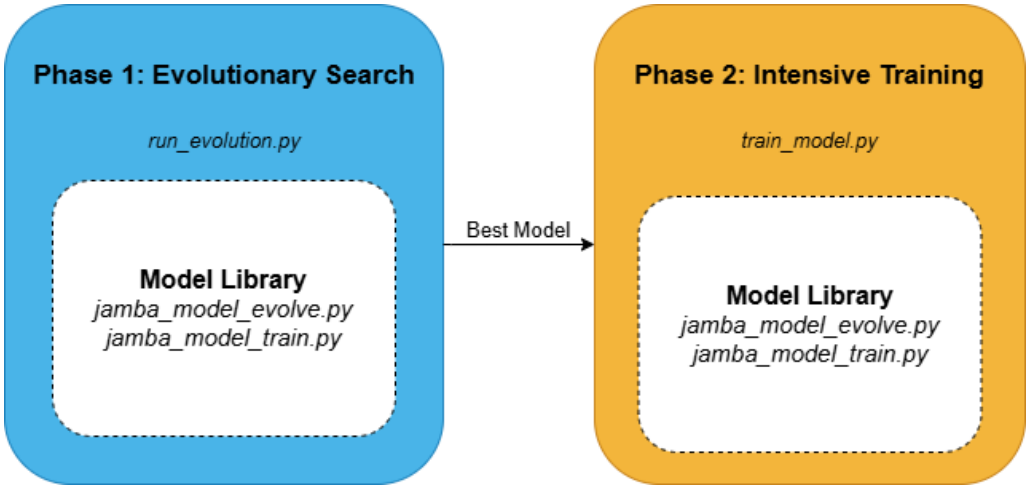

Figure 5.1: System Architecture Overview.

- **Model Library** (`jamba_model_evolve.py` and `jamba_model_train.py`) – contains all PyTorch module definitions: the Mamba block, the Transformer attention block, the hybrid Jamba layers, the Mixture-of-Experts (MoE) block, and the classification head. It also provides utility functions to load the

Mini-Jamba configuration from Hugging Face (`get_pretrained_config`) and to load a full pretrained model with original layer pattern (`from_pretrained`). These files are an adaptation of the `pytorch_mamba` repository[1], which itself is a simplified PyTorch reimplementation of the official Jamba model. We modified the library only to accept a variable-length binary genotype that controls the sequence and type of layers, enabling evolutionary exploration without altering the internal computation of the blocks.

- **Phase 1 — Evolutionary Search** (`run_evolution.py` and `run_evolution_propor.py`) – orchestrates the generational loop, genetic operators, population management, and coordinates the short-budget training of each individual. The engine is independent of the specific task; only the dataset-loading function and hyper-parameters (batch size, number of classes, etc.) differ between the AG News and PROPOR versions. Inside this phase, the functions `train_model` and `evaluate_model` handle the fixed-budget training and fitness calculation.
- **Phase 2 — Intensive Training** (`train_model.py` and `train_model_propor.py`) – after evolution finishes, the best genotype and its partially trained weights are saved to a checkpoint. These scripts reload that checkpoint, reconstruct the architecture using the model library, and fine-tune it on the full dataset for several epochs with a lower learning rate and a linear warm-up schedule.

The data flow during the evolutionary phase (Phase 1) is as follows:

1. The evolution engine generates a random genotype (a binary list).
2. The genotype is passed to the model library, which constructs a `JambaLM` model and wraps it in a `JambaClassifier`.
3. The training pipeline trains the classifier for a fixed number of steps using a generation-specific data seed, then returns the F1-score as fitness.
4. The evolution engine updates the population with selection, crossover, and mutation, logs the generation statistics, and repeats.

The intensive training phase (Phase 2) works as follows:

1. The saved checkpoint (containing the best genotype and its trained weights) is loaded.
2. The model library rebuilds the corresponding architecture.
3. The resulting classifier is trained on the full training set using a lower learning rate and a linear warm-up schedule, and the final model is evaluated on the test set.

[1] `https://github.com/Maxtimer97/pytorch_mamba`

The mapping from a binary genotype to a trainable PyTorch model is implemented in the `JambaLM` and `Jamba` classes of the model library.

The constructor `Jamba.__init__` receives a `JambaLMConfig` and a genotype list. It iterates over each gene index i:

- The layer position determines whether a Mixture-of-Experts variant is used, according to the fixed `expert_layer_offset` and `expert_layer_period` from the Mini-Jamba configuration. When the index satisfies $(i - \text{offset}) \bmod \text{period} = 0$, the layer is instantiated with multiple experts; otherwise, a single dense MLP is used.
- If the gene is 0, a `MambaLayer` is appended.
- If the gene is 1, an `AttentionLayer` is appended.

The resulting `nn.ModuleList` becomes the backbone of the model. The `JambaLM` class then combines this backbone with a static embedding layer, a final RMSNorm, and a linear language-model head. For classification, we override the head with a `JambaClassifier` that applies mean-pooling over the hidden states and projects to the required number of classes.

This design ensures that the evolutionary operators manipulate only the layer sequence, while the internal configuration of each layer (hidden size, expansion factor, number of attention heads, etc.) remains constant and equal to that of Mini-Jamba.

5.2 Evolutionary Engine and Short-Budget Training

The script `run_evolution.py` (and its PROPOR counterpart) implements the main evolutionary loop. The core function `evolve` receives the Mini-Jamba configuration and the task datasets and performs the following steps:

1. **Initialisation:** 30 random genotypes are generated (with lengths between 4 and 20). For each genotype, `evaluate_individual` is called, which builds the model, trains it for a fixed number of steps, and returns its fitness.
2. **Generation cycle:** For each generation, the population is sorted by fitness; the top individual is kept unchanged. For the remaining slots, parents are selected via tournament selection (size 3), crossed over, and mutated.
3. **Mutation:** Two independent operators are applied probabilistically. Bit-flip mutation (probability 0.5) flips each gene with probability $1/L$. Structural mutation (probability 0.25) either inserts a new random gene or deletes an existing one, with equal chance.
4. **Evaluation:** Each new child is evaluated with the same training pipeline, using a generation-specific data loader seed to guarantee fair intra-generation comparison.

5. **Logging:** After each generation, the population statistics (fitness, depth, proportion of Mamba layers) are appended to a CSV file, and for the first generation a convergence plot is saved.

6. **Checkpointing:** The best genotype and its trained weights are saved to a `.pt` file at the end of the run.

The function `train_model` (defined in the evolution scripts) executes the training of a single individual. Its design features are:

- A fresh DataLoader is created with `shuffle=True` and a generator initialised with a generation-specific seed (`DATALOADER_BASE_SEED + generation_index`). This ensures all individuals in the same generation see exactly the same sequence of mini-batches.
- Training runs for a fixed number of optimizer steps, using the AdamW optimizer with a constant learning rate of 4×10^{-4}.
- Every n steps, the model is evaluated on a validation set. If the validation F1 does not improve for three consecutive checks, training is stopped early. The best weights are restored before returning.
- The function returns the training time and the list of training losses (for plotting).

The validation function `evaluate_model` performs inference on the validation set without random shuffling, guaranteeing identical evaluation conditions for all individuals.

5.3 Intensive Training and Data Handling

The scripts `train_model.py` and `train_model_propor.py` perform the final intensive training of the best architecture discovered during evolution. They share the same structure:

- Load the checkpoint containing the genotype and the partially trained weights.
- Build the model with that genotype and the Mini-Jamba configuration.
- Train for several epochs on the full training set using a lower learning rate (3×10^{-5}) and a linear warm-up schedule.
- Evaluate on the official test split.

These scripts also produce CSV logs and save the best model weights for later use.

The datasets (AG News and PROPOR) are loaded using the Hugging Face `datasets` library. For AG News, 60,000 training samples and 1,500 validation samples are used during evolution. For PROPOR, the official training split is divided into 90% for training and 10% for validation. All texts are tokenized with the Mini-Jamba tokenizer, truncated to 128 or 512 tokens respectively, and packed into PyTorch tensors.

To support the fixed-seed mechanism, the data is loaded and split once before the evolution starts, producing static arrays of tokenized inputs. The DataLoader then permutes these arrays in a reproducible order using the seeded generator.

The system produces the following outputs for each independent run:

- `results_run_<n>_seed_<s>_<steps>_steps.csv` – generation-by-generation log of fitness, F1, latency, parameter count, depth, and genotype of every individual.
- `best_model_run_<n>_seed_<s>_<steps>_steps.pt` – a checkpoint containing the genotype, the state dictionary of the trained classifier, the best F1, the random seed, and the generation where it was found.
- `trained_model_results_run_<n>_seed_<s>_<steps>_steps.csv` – after intensive training, logs of test performance.
- `trained_model_run_<n>_seed_<s>_<steps>_steps.pt` – the final best weights from intensive training.

All output files follow a consistent naming scheme that encodes the run index, random seed, and number of evolutionary training steps, ensuring reproducibility and easy identification.

Chapter 6

Implementation

This chapter describes the concrete technologies, code organization, and development workflow that were used to realize the system presented in Chapter 5. The goal is to provide enough detail for an independent researcher to understand how the experiments were carried out and, if desired, to reproduce the main results. The full source code and all experiment logs are publicly available at `https://github.com/filipeobrist/LLM_Evolution`.

6.1 Codebase and Core Implementation

The entire codebase is written in **Python 3.8** and relies on the following main libraries:

- **PyTorch 1.13+** – deep learning framework for model definition, training, and inference.
- **Transformers (Hugging Face)** – `AutoTokenizer` and `AutoModelForCausalLM` for loading the Mini-Jamba tokenizer and, when needed, the pretrained model.
- **Datasets (Hugging Face)** – loading of AG News and PROPOR corpora.
- **scikit-learn** – metrics (F1-score, accuracy, precision, recall) for evaluation.
- **Matplotlib, NumPy, Pandas** – data visualisation and logging.
- **mamba_ssm** – provides the official hardware-optimized CUDA kernel for the Mamba selective scan. The `MambaBlock` configuration sets `use_cuda = True` so that all Mamba layers benefit from the fused, memory-efficient implementation.

All experiments were run on an NVIDIA GeForce RTX 3080 Ti GPU with 12 GiB of VRAM, using CUDA 12.2 and driver 535.183.01.

The project is organised into a single Python package and two main data folders. Figure 6.1 shows the top-level layout.

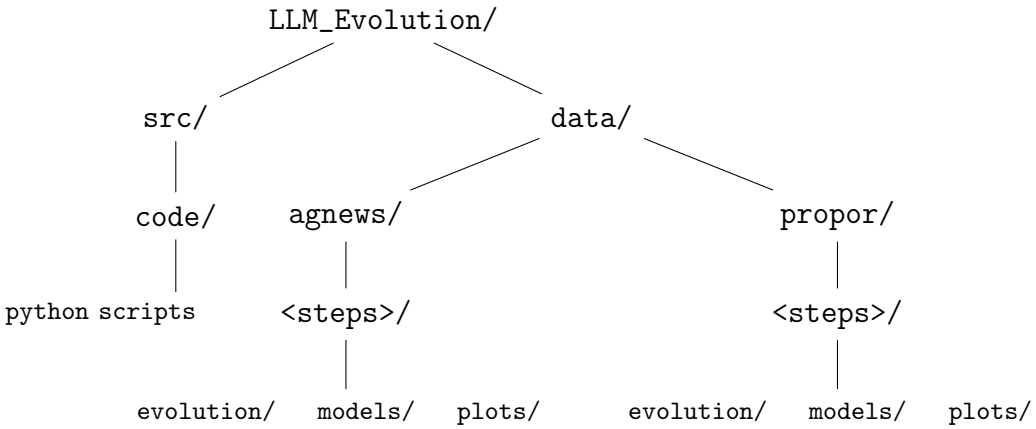

Figure 6.1: Project directory structure. The <steps> placeholder represents a subfolder named after the number of evolutionary training steps (e.g., 100_steps).

Code (src/code/)

- **Model library:** jamba_model_evolve.py and jamba_model_train.py contain all neural module definitions.
- **Evolution:** run_evolution.py (AG News) and run_evolution_propor.py (PROPOR) drive the evolutionary search.
- **Intensive training:** train_model.py (AG News) and train_model_propor.py (PROPOR) fine-tune the best evolved genotype on the full dataset.
- **Arbitrary genotype training:** train_jamba.py accepts a genotype string and trains the corresponding architecture from scratch. It was used to train the 32-layer Jamba-pattern baseline (JambaGen) and can be employed for any custom genotype.
- **Memory testing:** max_layer.py builds a model with a given number of layers and a realistic batch, then reports the peak GPU memory usage. This script was run before starting an evolution to ensure the chosen batch size and sequence length fit within 12 GiB.
- **Latency:** latency.py measures inference latency and peak memory for a saved model.
- **Plotting:** plot_results.py reads the evolution CSV logs and generates the figures presented in Chapter 7.

Data (data/) The data/ folder is organised by task (AG News vs. PROPOR) and, within each task, by the number of evolutionary training steps (e.g., 100_steps). Each step subfolder contains:

- evolution/ – CSV logs from the evolutionary runs and the checkpoints (best_model_*.pt).

- `models/` – CSVs and checkpoints from the post-evolution intensive training.
- `plots/` – output figures of the plotting scripts.

This structure keeps all artefacts belonging to a particular experiment (task + step budget) together and makes it easy to locate the results of any configuration.

The Mamba selective scan used in this dissertation is the official CUDA-based implementation provided by the `mamba_ssm` library. The `MambaBlock` configuration is set to `use_cuda = True`, so the forward pass calls the fused `selective_scan_cuda` kernel that performs the convolution, activation, selective scan and output projection in a single highly optimised operation. This ensures that the Mamba layers operate at their full intended efficiency, making the latency and memory measurements reported in Chapter 7 directly comparable to the Transformer baselines.

The decoding logic (described in Chapter 5) is implemented in the constructor of the `Jamba` class. A genotype list is directly mapped to a sequence of `MambaLayer` and `AttentionLayer` objects, with Mixture-of-Experts placement determined by the fixed `expert_layer_offset` and `expert_layer_period` from the Mini-Jamba configuration. This decoding is called from `evaluate_individual` (evolution) and from the intensive training scripts.

To guarantee that every individual within a generation is trained on exactly the same sequence of mini-batches, the training function creates a `torch.Generator` initialised with `DATALOADER_BASE_SEED + generation_index`. The validation set is always traversed in its original order (`shuffle=False`). This mechanism is implemented in `train_model` and `evaluate_model` inside the evolution scripts.

6.2 Configuration and Experimental Runs

Table 6.1 lists the hyper-parameter values used during the evolutionary search, while Table 6.2 summarizes the settings for the post-evolution intensive training.

Table 6.1: Hyper-parameters for the evolutionary search.

Parameter	**AG News**	**PROPOR**
Population size	30	30
Number of generations	100	100
Min. / max. layers	4 / 20	4 / 20
Crossover rate	0.8	0.8
Bit-flip mutation prob.	0.5	0.5
Structural mutation prob.	0.25	0.25
Training steps	100, 300, 400	400
Batch size	16	4
Learning rate	4×10^{-4}	4×10^{-4}
Max. sequence length	128 tokens	512 tokens

Table 6.2: Hyper-parameters for the intensive (post-evolution) training.

Parameter	**AG News**	**PROPOR**
Learning rate	3×10^{-5}	3×10^{-5}
Epochs	4	8
Batch size	32	16
Max. sequence length	128 tokens	512 tokens

For each configuration we ran several independent evolutionary runs with different random seeds. The seeds used across all experiments were `42, 123, 999, 2024, 7, 2026, 22` (seven seeds). Table 6.3 reports the average wall-clock time per run for each setting, obtained by summing the per-generation time (`gen_time_min`) across the 100 generations of a run.

Table 6.3: Average total evolution time per run for each configuration.

Task	**Training steps**	**Batch size**	**Avg. time per run (h)**
AG News	100	16	`9.96`
AG News	300	16	`23.48`
AG News	400	16	`24.55`
PROPOR	400	4	`57.76`

The intensive training (post-evolution) took approximately 2.5 hours per run for AG News (4 epochs, batch 32) and 5.5 hours for PROPOR (8 epochs, batch 16).

6.2.1 Latency Measurement

The inference latency of the final models (after intensive training) was measured with the script `latency.py`. For each model, we:

- Loaded the saved checkpoint and reconstructed the architecture with the given genotype.
- Set the model to evaluation mode.
- Created a random input tensor with the same sequence length used during training (128 for AG News, 512 for PROPOR) and a batch size of 1.
- Performed 50 warm-up forward passes, then timed 500 additional forward passes using CUDA events.
- Recorded the mean latency, as well as the peak GPU memory usage.

All latency measurements were performed on the same hardware (RTX 3080 Ti) with the official CUDA-based Mamba kernel active.

6.2.2 Reproducibility and Memory Management

To ensure reproducibility, each run fixes the random seed of Python, NumPy, and PyTorch using the function `set_seed`. CUDA deterministic mode is enabled (`torch.backends.cudnn.deterministic = True`), and the data loader seed is derived from a base constant (`DATALOADER_BASE_SEED`) plus the generation index.

The evolution loop automatically logs every generation to a CSV file, and the best model is saved to a checkpoint. The naming scheme of all output files includes the run index, the random seed, and the number of evolutionary training steps, making it straightforward to trace any experimental result back to its configuration.

To reproduce the main results, one can execute:

```
python run_evolution.py          # AG News evolution
python train_model.py            # intensive training (AG News)
python run_evolution_propor.py   # PROPOR evolution
python train_model_propor.py     # intensive training (PROPOR)
```

The scripts require the Hugging Face datasets to be accessible online and the Mini-Jamba tokenizer/model to be downloaded (or cached) from Hugging Face.

Because the evolutionary search evaluates thousands of models, careful memory management was essential. After each fitness evaluation, the model is deleted and the GPU cache is emptied (`gc.collect(); torch.cuda.empty_cache()`). Gradient clipping (`clip_grad_norm_`) and the chosen batch sizes help keep memory usage within the 12GiB limit.

The script `max_layer.py` was used before each experiment to determine the maximum feasible model size (number of layers, batch size, sequence length) that would fit in GPU memory. This informed the choice of `MAX_LAYERS = 20` and the batch sizes reported in Table 6.1. The official CUDA kernel used for the Mamba scan is already memory-efficient; the main memory bottleneck lies in the large activation tensors created by the parallel scan algorithm itself, which limits the maximum sequence length and batch size.

Chapter 7

Results

This chapter presents the experimental data obtained from the evolutionary runs and the subsequent intensive training on the two text classification tasks, AG News (English) and PROPOR (Portuguese scientific domain). The section is organised as follows: first we illustrate the evolutionary dynamics that characterised the search process, then we report the final performance of the evolved architectures on each task, together with comparisons against the manually designed baselines.

7.1 Evolutionary Dynamics

To understand how the population evolved over the course of the search, we aggregated the results of all independent runs (seven seeds) for each configuration and computed, for every generation, the mean and the standard deviation of the following metrics: population mean fitness, best fitness, average depth, and the proportion of Mamba (gene 0) versus Transformer (gene 1) layers. Results are shown for the three different step budgets used in the AG News experiments (100, 300, and 400 steps) and for the 400-step PROPOR runs. The corresponding plots for the PROPOR task are provided in Appendix A.3 for completeness.

7.1.1 Fitness progression

Figure 7.1 shows the evolution of the mean population fitness and the fitness of the best individual, separately for each step budget. The shaded band represents $\pm$ one standard deviation across the seven seeds. In all configurations the best fitness rises quickly and then levels off after approximately 40 generations.

7.1.2 Architectural convergence

The average depth of the population over time is shown in Figure 7.2. In all three step budgets the depth drops from the initial range (4-20 layers) to about 5 layers

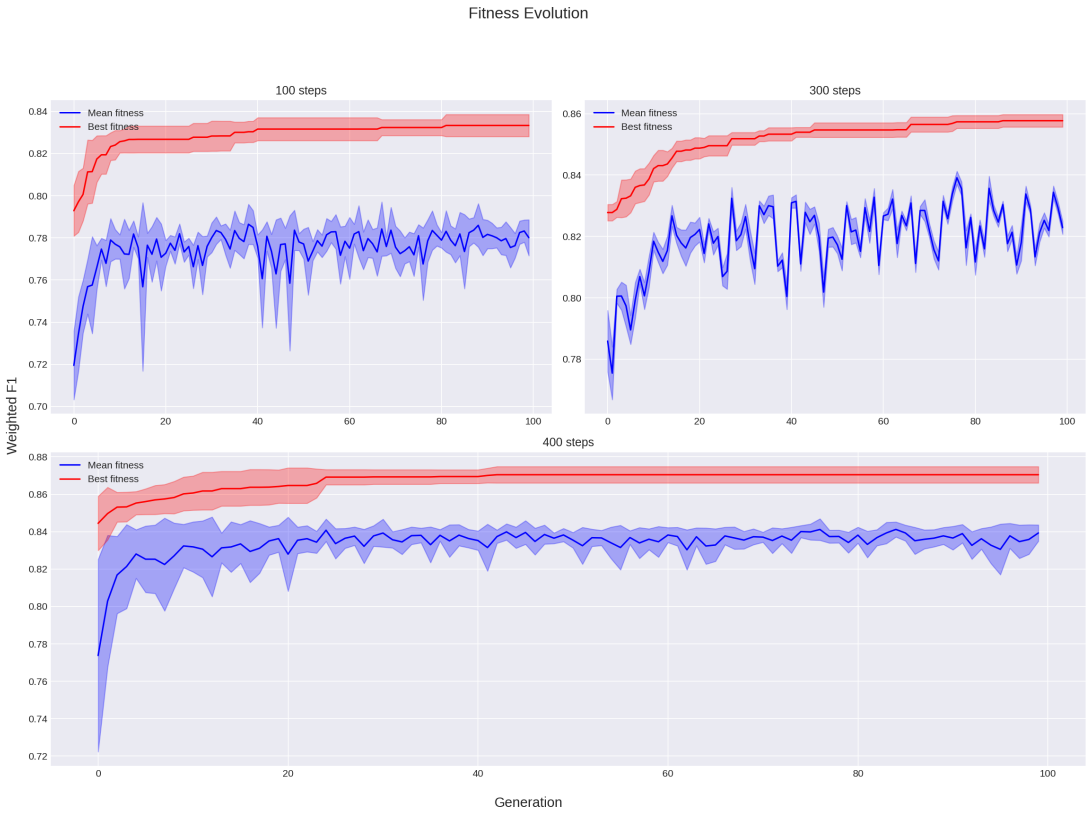

Figure 7.1: Mean and best fitness (F1) across generations, for AG News with 100, 300, and 400 evolutionary training steps. Each panel aggregates seven independent seeds.

within the first 20 generations and then remains stable. The same behaviour was observed for the PROPOR task.

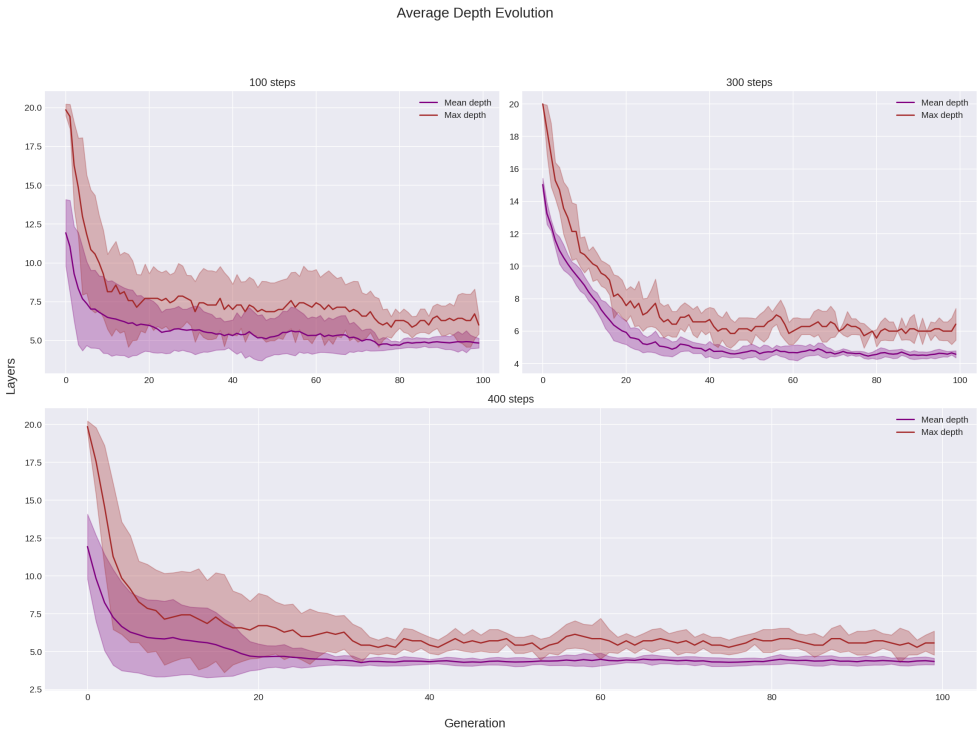

Figure 7.2: Average population depth per generation for AG News with 100, 300, and 400 steps.

The proportion of Mamba and Transformer layers over time is shown in Figure 7.3. In the shortest training budget (100 steps) the population quickly becomes dominated by Mamba layers (80 %), whereas with more steps (300 and 400) the proportion of Transformer layers remains slightly higher, although Mamba still dominates.

7.1.3 Discriminative power of the fitness signal

To verify that the limited training budget was sufficient to produce a meaningful fitness signal, we examined the cross-entropy loss of each individual in generation 0 of the AG News runs with 100 steps (the smallest budget used). As shown in Figure 7.4, the training loss of each individual decreases rapidly over the 100 steps and begins to converge toward a stable value, indicating that the model is genuinely learning from the data. This confirms that the limited training budget produces a meaningful fitness signal rather than one dominated by random noise or initialization effects.

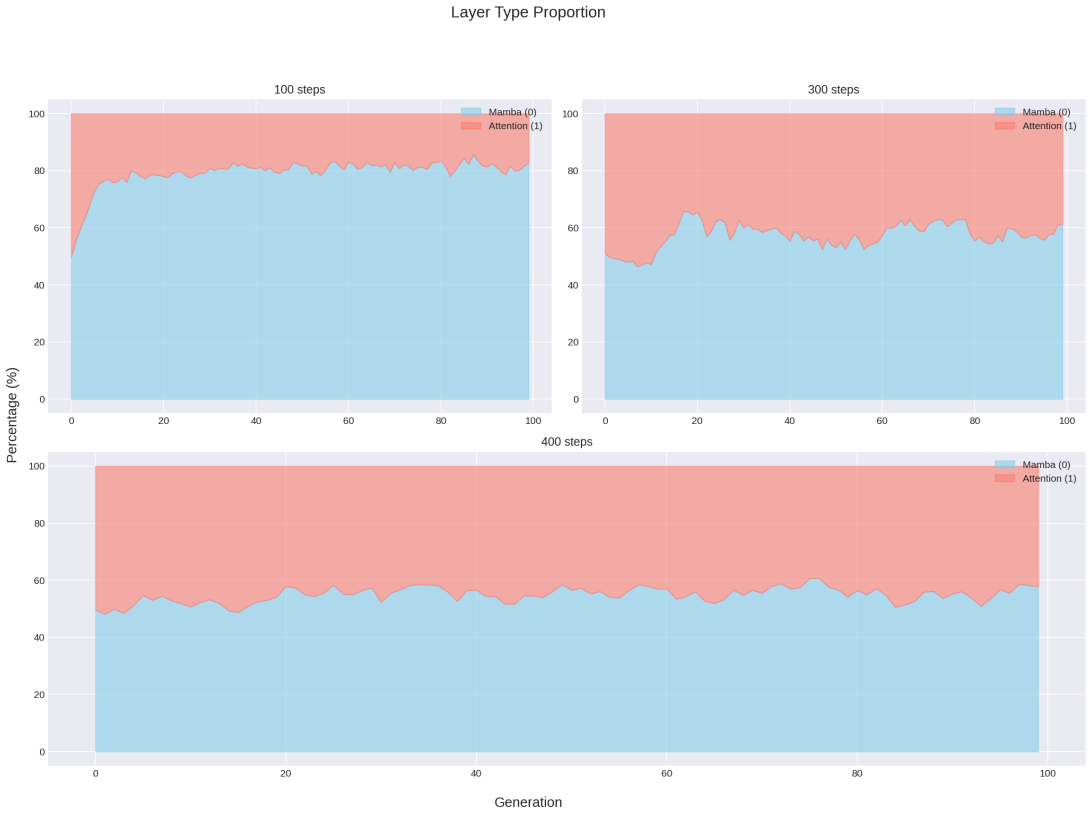

Figure 7.3: Proportion of Mamba (0) and Transformer (1) layers in the population, for AG News with 100, 300, and 400 steps.

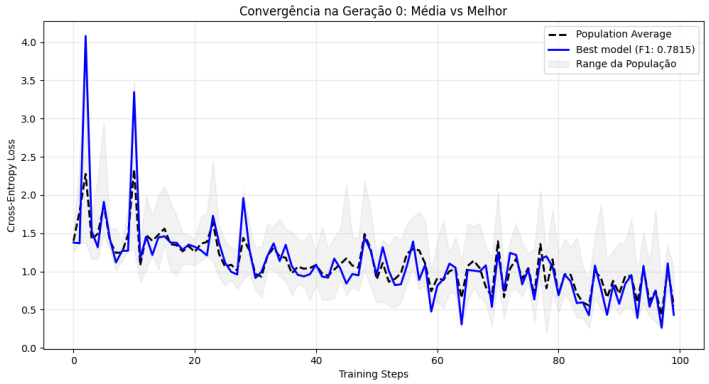

Figure 7.4: Cross-entropy loss of the 30 individuals in generation 0 (AG News, 100 steps).

7.2 AG News Results

After intensive training of the best genotype from each evolutionary run, we evaluated the final models on the official AG News test set. Table 7.1 summarizes the results grouped by the number of evolutionary training steps; for each group we report the mean and standard deviation of the F1-score, accuracy, precision, recall, and inference latency. The complete per-seed breakdown for all AG News experiments is provided in Appendix A.1.

Table 7.1: Performance of the evolved architectures on AG News, grouped by evolutionary training steps. Each group aggregates seven independent runs; values are reported as mean (standard deviation). Legend: Acc = Accuracy, Prec = Precision, Rec = Recall, Lat = Latency (ms per document).

Experiment	**F1**	**Acc**	**Prec**	**Rec**	**Lat. (ms)**
E1 - 100 steps	**92.31** (0.11)	**92.34** (0.11)	**92.31** (0.11)	**92.31** (0.11)	8.85 (0.37)
E2 - 400 steps	92.09 (0.24)	92.13 (0.23)	92.11 (0.20)	92.09 (0.24)	**7.54** (0.89)
E3 - 300 steps	92.07 (0.13)	92.06 (0.13)	92.07 (0.13)	92.07 (0.13)	7.71 (0.80)

The three best architectures (one per experiment) are listed in Table 7.2, together with their individual metrics. The best genotypes are extremely compact, consisting of 4 or 5 layers, and are predominantly Mamba.

Table 7.2: Best architecture discovered for each AG News experiment. Legend: Acc = Accuracy, Prec = Precision, Rec = Recall

Genotype	**Params**	**F1**	**Acc**	**Prec**	**Rec**
`[0,0,0,0,0] (E1)`	44.5M	**92.4**	**92.4**	**92.5**	**92.4**
`[0,1,0,1] (E2)`	42.5M	92.3	92.3	92.4	92.3
`[0,0,0,1] (E3)`	43M	92.2	92.2	92.2	92.2

Table 7.3 compares these three best architectures with the manually designed baselines. The evolved models match the Mini-Jamba baseline and outperform the deeper JambaGen trained from scratch, while remaining behind the pre-trained Transformer baselines.

Table 7.3: Comparison of the best evolved architectures (one per experiment) with baselines on AG News. Legend: Acc = Accuracy, Prec = Precision, Rec = Recall

Model	**Params**	**F1**	**Acc**	**Prec**	**Rec**
DistilBERT (pre-trained)	66M	**94.3**	**94.3**	**94.3**	**94.3**
BERT (pre-trained)	110M	93.8	93.8	93.8	93.8
Mini-Jamba (pre-trained)	69.5M	92.5	92.5	92.5	92.5
JambaGen (from scratch)	110M	92.1	92.1	92.1	92.1
Evolved (best E1)	44.5M	92.5	92.5	92.4	92.4
Evolved (best E2)	44.0M	92.3	92.3	92.3	92.3
Evolved (best E3)	42.5M	92.3	92.3	92.4	92.3

7.3 PROPOR Results

The same procedure was applied to the PROPOR scientific field classification task. All runs used 400 evolutionary training steps and batch size 4. Table 7.4 reports the mean and standard deviation of the performance metrics across the seven independent runs. The full per-seed results for PROPOR can be found in Appendix A.2.

Table 7.4: Performance of the evolved architectures on PROPOR (7 runs, 400 steps, batch 4).

Metric	**Mean**	**Std**
Accuracy	88.10	1.32
Precision	83.80	1.66
Recall	83.20	1.99
F1	83.44	1.74
Latency (ms)	9,38	1.77

The best architecture found for PROPOR was `[0,0,0,0]` (4 Mamba layers, 43.4 M parameters), achieving a F1 of 85.0. Table 7.5 compares this model with the fine-tuned AMALIA baseline.

Table 7.5: Best evolved PROPOR model versus the AMALIA baseline. Legend: Prec = Precision, Rec = Recall

Model	**Params**	**F1**	**Prec**	**Rec**
AMALIA (fine-tuned)	9B	84.1	**84.5**	83.7
Evolved (best)	43.4M	**85.0**	84.7	**85.4**

Chapter 8

Discussion

This chapter interprets the experimental results presented in Chapter 7 in the light of the research questions formulated in Chapter 1 and the methodological constraints described in Chapter 4. It also identifies the main limitations of the study, discusses potential threats to validity, and highlights unexpected observations that emerged during the investigation. The final conclusions and directions for future work are deferred to Chapter 9.

8.1 Interpretation of Results

8.1.1 Effectiveness of Evolutionary Search (RQ1)

The results demonstrate that an evolutionary algorithm with a simple binary genotype and standard genetic operators can successfully navigate a heterogeneous architecture space that mixes Transformer and Mamba layers. Across all configurations and both tasks, the search consistently improved the population fitness over generations, and the final evolved architectures were always valid, trainable models. This provides a clear affirmative answer to RQ1: *EC can be used to evolve hybrid Transformer-Mamba architectures.*

However, the search exhibited a strong and consistent preference for shallow, Mamba-only or Mamba-dominant designs. Only a handful of Transformer layers survived in the best genotypes, and those appeared almost exclusively when the evolutionary training budget was larger (300–400 steps). This indicates that, under the constrained evaluation regime, Mamba layers offer a faster convergence path and thus a higher fitness after a short training period. The search was therefore effective at finding architectures that are efficient to train, but it did not discover genuinely hybrid topologies that interleave both layer types in a balanced way.

The tendency towards shallowness was also robust: even when the initial population was forced to contain at least 10 layers, the average depth dropped to about 5 layers within the first 20 generations. This convergence pattern, observed in all

runs, suggests that the fitness landscape is heavily biased towards compact architectures, at least when training budgets are limited. It is plausible that deeper networks would require more training steps to outperform the shallow ones, and the current evaluation protocol simply does not give them enough time.

It is also worth noting that the task itself may not require large models. The best overall performance on AG News was achieved by a 66M-parameter DistilBERT, and the 44M-parameter evolved models reached very close to that ceiling. In this context, the evolutionary search appears to have correctly identified that additional capacity brings little benefit, and the convergence towards small architectures may reflect a genuine property of the problem rather than a failure of the algorithm.

8.1.2 Competitiveness of Evolved Architectures (RQ2)

Concerning RQ2, the evolved architectures demonstrated competitive performance under fair comparison conditions. On AG News, they matched the fine-tuned Mini-Jamba baseline (69.5M parameters) while using approximately 40% fewer parameters, and they outperformed the much deeper JambaGen (110M) trained from scratch under the same protocol. This shows that the evolutionary search can identify parameter-efficient models that achieve a similar accuracy level to a manually designed hybrid architecture, without relying on pre-training.

Nevertheless, a gap of about 2 percentage points in F1-score remains between the evolved models and the heavily pre-trained Transformer baselines (DistilBERT, BERT). This gap is expected and highlights the fundamental role of large-scale pre-training, which was completely absent from the evolved architectures. The comparison is not entirely balanced: the Transformer baselines benefit from extensive pre-training on massive text corpora, whereas the evolved models were trained solely on the task-specific data. Viewed in this light, the achieved performance is encouraging.

The PROPOR results further strengthen this picture. The best evolved model achieved a F1-score of 85.0, surpassing the fine-tuned AMALIA baseline (84.1) while using five orders of magnitude fewer parameters. This outcome underscores the potential of automated architecture design for domain-specific tasks, especially when large pre-trained models are unavailable or impractical to deploy.

8.1.3 Influence of the Training Budget

A somewhat unexpected finding was that increasing the number of evolutionary training steps did not lead to a corresponding improvement in the final performance after intensive training. The three AG News experiments (100, 300, 400 steps) produced final architectures whose average F1-scores were essentially indistinguishable (92.31, 92.07, 92.09). This suggests that the evolutionary search converges to a plateau of similarly capable architectures early on, and the in-

tensive post-evolution training largely compensates for the short fitness evaluation. It also implies that the additional computation spent on longer evolutionary training did not translate into better final models, raising questions about the optimal allocation of resources between the search and the final training phases.

8.1.4 Latency and Efficiency

The latency measurements revealed that architectures containing Transformer layers were consistently faster than pure-Mamba ones, despite the official CUDA-based Mamba kernel being active. At the sequence lengths used in our tasks (128–512 tokens), the highly optimized attention kernels (FlashAttention) still hold a constant-factor advantage over the Mamba scan. This finding is consistent with other reports in the literature and does not contradict the theoretical linear-time complexity of Mamba, which would become apparent at much longer sequence lengths. It does, however, serve as a reminder that theoretical efficiency does not always translate into practical speed at typical classification scales.

The evolved architectures are notably parameter-efficient: they achieve almost the same accuracy than Mini-Jamba with significantly fewer parameters, and they approach the performance of much larger pre-trained models while being entirely trainable from scratch on a single consumer-grade GPU. This efficiency is a direct outcome of the evolutionary search, which explicitly selects for architectures that learn quickly under a limited budget.

8.2 Limitations

Several limitations of the present work should be acknowledged.

- **Training budget during evolution.** The fitness of each individual was evaluated after only 100–400 optimizer steps. With a batch size of 16 and a training subset of 60 000 samples (AG News), this means each individual saw between approximately 1 600 and 6 400 unique examples, corresponding to roughly 2.7%–10.7% of an epoch over the available training data. Such a limited exposure is far from a complete training, and it heavily biases the search towards architectures that converge quickly, rather than those with the highest ultimate potential.
- **Sequence length.** The maximum sequence length was capped at 128 tokens for AG News and 512 tokens for PROPOR, due to GPU memory constraints. These lengths are sufficient for the chosen datasets but do not allow the exploitation of Mamba's linear-complexity advantage over Transformers, which only becomes significant for sequences of several thousand tokens.
- **Search space granularity.** The search was limited to the depth and the layer type (Transformer vs. Mamba). Internal hyper-parameters such as the number of attention heads, the state dimension, the expansion factor, and the

placement of Mixture-of-Experts layers were fixed. A richer search space could yield more diverse and potentially better architectures.

- **Weight inheritance.** No mechanism for transferring trained weights from parents to offspring was implemented. Although this was a deliberate choice to avoid biasing the search, it also means that every new individual had to be trained from scratch, increasing the computational cost and possibly reducing the effective exploration of the space.
- **Number of tasks and seeds.** The experiments were conducted on only two text classification tasks and with seven independent runs per configuration. While seven runs provide a reasonable estimate of variability, a larger number of seeds and a wider variety of tasks would strengthen the generalizability of the conclusions.

8.3 Threats to Validity

Internal validity. To ensure fair comparisons among individuals of the same generation, the data loader was seeded with a generation-specific value, so that all architectures were trained on exactly the same mini-batches. Early stopping was applied consistently, and the same optimizer and learning rate were used throughout. These measures reduce the risk that differences in fitness are due to random data ordering or training irregularities. However, the use of a small validation set (1 500 samples for AG News, a 90/10 split for PROPOR) may introduce some noise in the fitness estimates.

External validity. The study is limited to text classification on news articles and scientific abstracts. It is unclear to what extent the evolved architectures would transfer to other NLP tasks such as question answering, generation, or reasoning. Additionally, only English and Portuguese were studied; the behaviour in other languages remains unexplored.

Construct validity. The fitness function was the F1-score on a validation set. While this metric is standard for classification, it may not perfectly capture the true quality of an architecture, especially when the validation set is small.

Conclusion validity. The standard deviation of the final performance across seeds was small (0.11–0.24 F1 points for AG News), indicating that the evolutionary process is reasonably stable. Nevertheless, the number of independent runs (7) is at the lower bound for drawing strong statistical conclusions. The findings should therefore be interpreted as indicative rather than definitive.

8.4 Unexpected Observations

Beyond the anticipated results, several observations merit attention:

- The dominance of Mamba layers and the absence of true hybrid architectures were more extreme than initially expected. Even with the structural mutation operators that could insert Transformer layers, the population overwhelmingly converged to Mamba-only designs. This raises the question of whether attention layers are genuinely less suitable for these tasks, or whether the evaluation protocol simply does not allow them to demonstrate their value.
- The initial-depth experiment (forcing a minimum of 10 layers) had no lasting effect on the final architecture. This suggests that the fitness landscape exerts a very strong attraction towards shallow models, which cannot be counteracted by simply modifying the starting population.
- Despite the theoretical efficiency of Mamba, the observed inference latency was consistently higher for Mamba-heavy architectures than for Transformer-heavy ones, even with the official CUDA kernel enabled. This contradicts the intuitive expectation that Mamba should always be faster, and illustrates the importance of implementation-level optimizations and the specific sequence-length regime.
- The fact that longer evolutionary training did not improve the final performance after intensive training was counter-intuitive. It implies that the evolutionary search acts more as a filter that removes very poor architectures early on, rather than as a fine-grained optimizer that continuously refines them. Once a sufficiently good architecture is found, the post-evolution intensive training dominates the final outcome.

The next chapter summarizes the main contributions of this dissertation and outlines concrete directions for addressing the limitations discussed here.

Chapter 9

Conclusion and Future Work

This final chapter summarizes the main contributions of the dissertation, revisits the initial research questions in light of the experimental evidence, and outlines promising directions for future investigation. The limitations of the current work, already discussed in detail in Chapter 8, are only briefly recalled here.

9.1 Summary of Contributions

This dissertation set out to investigate whether Evolutionary Computation could be applied to the automatic design of hybrid Transformer–Mamba architectures for text classification. The main contributions can be summarised as follows:

- **An evolutionary framework for hybrid architecture search.** We designed and implemented a genetic algorithm that operates on variable-length binary genotypes, where each gene determines whether a layer is a Mamba block or a Transformer block. The framework includes custom crossover, bit-flip mutation, and structural mutation operators, and it evaluates individuals through a short, fixed-budget training on a subset of the target dataset. The entire pipeline was built in PyTorch, using the official CUDA-based Mamba kernel for efficiency.
- **A systematic experimental study across two tasks.** We applied the evolutionary search to two text classification tasks with different characteristics: AG News (English, 4 classes, short texts) and PROPOR (Portuguese scientific domain, 5 classes, longer texts). For each task, multiple independent runs (seven seeds) were performed under different training budgets (100–400 evolutionary steps), and the best discovered architectures were further subjected to intensive fine-tuning on the full training set.
- **Competitive results with parameter-efficient models.** The evolved architectures consistently matched the Mini-Jamba baseline while using approximately 40% fewer parameters, and they outperformed a deeper Jamba-style hybrid trained from scratch. On the PROPOR task, the best evolved model

exceeded the F1-score of a fine-tuned 9B-parameter AMALIA model, despite being more than 200 times smaller.

- **Insights into architectural search dynamics.** We provided empirical evidence that the evolutionary search converges rapidly to shallow, Mamba-dominated architectures, that this preference is robust to the initial population depth, and that a longer evolutionary training budget does not necessarily lead to better final architectures after intensive post-evolution training.

9.2 Revisiting the Research Questions

RQ1 — Can we use EC to evolve heterogeneous Transformer-Mamba architectures?

Yes. The proposed evolutionary algorithm proved capable of exploring the heterogeneous search space and producing valid, trainable models under realistic computational constraints. The variation operators (crossover, bit-flip mutation, structural insertion/deletion) were sufficient to navigate the space, and the fixed-seed evaluation protocol ensured fair comparisons among individuals.

However, the search consistently converged to shallow, Mamba-only or Mamba-dominant architectures. While this demonstrates the effectiveness of EC in identifying efficient designs for the given training budget, it also shows that the current evaluation setup strongly favours architectures that learn quickly. True hybrid topologies, where Transformer and Mamba layers are interleaved in a balanced manner, remained largely absent from the final populations.

RQ2 — Can evolutionary search discover hybrid architectures with competitive performance compared to manually designed baselines?

Yes, to a significant extent. The evolved architectures were competitive with hand-crafted baselines of comparable size. They matched the Mini-Jamba baseline with fewer parameters, outperformed a larger Jamba-style model trained from scratch, and came close to heavily pre-trained Transformers despite having no pre-training whatsoever. The small gap to models like DistilBERT and BERT is explained by the enormous advantage that large-scale pre-training provides, and closing that gap without pre-training remains an open challenge.

The PROPOR results further demonstrate that, for domain-specific tasks where large pre-trained models may not exist, evolutionary search can produce remarkably efficient architectures that compete with models several orders of magnitude larger.

9.3 Broader Implications

Beyond the specific numerical results, this dissertation delivers a broader message: Evolutionary Computation can be a practical tool for architectural explo-

ration even in the era of large language models. While the current trend is to scale up a few manually designed architectures, our findings suggest that there is value in automating the search for efficient designs, particularly in resource-constrained settings or for specialised domains where custom architectures can outperform generic, massive models.

The fact that the search consistently discovered compact architectures also has implications for green AI: smaller, faster models require less energy for training and inference, and evolutionary methods can help identify such models in a principled way.

9.4 Future Work

The limitations discussed in Chapter 8 point to several concrete avenues for future research:

- **Extended training budget and longer sequences.** Increasing the number of training steps per individual, or even using multi-epoch training during evolution, would give larger and more complex architectures a fairer chance to demonstrate their potential. Similarly, testing on long-context tasks (e.g., document-level classification or summarisation with sequences of thousands of tokens) would better capture the theoretical advantage of Mamba over Transformers.
- **Enriched search spaces.** Allowing the evolutionary search to also optimise internal hyper-parameters—such as the number of attention heads, the state dimension, the expansion factor, or the expert placement—could lead to more diverse and possibly better architectures. Representations inspired by grammar-based evolution or NEAT could support more flexible topologies, including skip connections or non-sequential layer arrangements.
- **Weight inheritance and lifelong learning.** Incorporating a mechanism for transferring trained weights from parents to offspring (for example, through an intra-type layer matching strategy) could significantly accelerate the evolution and enable the discovery of deeper architectures. This would also open the door to continual architecture evolution, where models are refined over time as new data arrives.
- **Multi-objective optimisation.** Adding inference latency, memory consumption, or energy usage as explicit optimisation objectives would allow the search to directly target the most efficient architectures for deployment, rather than relying on a purely accuracy-based fitness.
- **Application to other tasks and modalities.** The same evolutionary framework could be applied to tasks beyond text classification, such as natural language inference, question answering, or even vision-language tasks where hybrid architectures may offer novel trade-offs.

- **Pre-training the evolved architectures.** The most direct path to closing the gap with Transformers like BERT and DistilBERT would be to take the best evolved architectures and pre-train them on large corpora, then fine-tune on the target tasks. This would quantify how much of the current gap is due to architecture alone and how much is due to the lack of pre-training.

In the end, this dissertation does not claim to have solved a grand challenge, nor to have produced a model that will change the world. It is, quite simply, an honest exploration of whether Evolutionary Computation can help design hybrid language models — and the answer turned out to be a quiet "yes". Watching a simple genetic algorithm, running on a single modest GPU, repeatedly discover compact architectures that held their own against far larger and more resource-intensive baselines was genuinely surprising. There is something hopeful in seeing that even with limited resources, evolution can find efficient solutions that we might not have thought to build by hand. The ambition was never to exhaustively explore the entire design space in a few months — an impossible task, especially when no prior work existed at this specific intersection of neuroevolution and hybrid sequence models. There was no foundation to build upon, no established pipeline to follow, no guarantee that the approach would even work. What this dissertation provides, then, is exactly that foundation: a launching zone from which more daring explorations can now take off.

References

[1] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 7(8):177–188, 2025. URL https://www.nature.com/articles/s42256-024-00975-8.

[2] Jay Alammar. The illustrated transformer, 2018. URL https://jalammar.github.io/illustrated-transformer/.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners, 2020. URL https://arxiv.org/abs/2005.14165. arXiv:2005.14165.

[4] Dikshit Chauhan, Bapi Dutta, Indu Bala, Niki van Stein, Thomas Bäck, and Anupam Yadav. Evolutionary computation and large language models: A survey of methods, synergies, and applications, 2025. URL https://arxiv.org/abs/2505.15741. arXiv:2505.15741.

[5] Krishna Chitty-Venkata, Arjun Somani, et al. A survey of neural architecture search. *ACM Computing Surveys*, 2022.

[6] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL https://arxiv.org/abs/2405.21060. arXiv:2405.21060.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2018. URL https://arxiv.org/abs/1810.04805. arXiv:1810.04805.

[8] Guodong Du, Jing Li, Hanting Liu, Runhua Jiang, Shuyang Yu, Yifei Guo, Sim Kuan Goh, and Ho-Kin Tang. Knowledge fusion by evolving weights of language models, 2024. URL https://arxiv.org/abs/2406.12208. arXiv:2406.12208.

[9] Wenqi Fan, Haohao Qu, and Tyler Derr. Training-free model architecture search for mamba. In *NeurIPS*, 2025. URL https://openreview.net/forum?id=8q2kReYRDn.

[10] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. Zamba: A compact 7b ssm hybrid model, 2024. URL https://arxiv.org/abs/2405.16712. arXiv:2405.16712.

[11] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL https://arxiv.org/abs/2312.00752. arXiv:2312.00752.

[12] John Timothy Halloran, Manbir S. Gulati, and Paul F. Roysdon. Mamba state-space models can be strong downstream learners. In *NeurIPS*, 2024. URL https://openreview.net/forum?id=C3t6GMPnC5.

[13] Mansoor Hamidzadeh. ag-news-bert-classification, 2023. URL https://huggingface.co/mansoorhamidzadeh/ag-news-bert-classification. Fine-tuned BERT base model on the AG News dataset.

[14] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. arXiv, 2016. URL https://arxiv.org/abs/1604.00772. arXiv:1604.00772.

[15] Wenjun Huang, Jiakai Pan, Jiahao Tang, Yanyu Ding, Yifei Xing, Yuhe Wang, Zhengzhuo Wang, and Jianguo Hu. Ml-mamba: Efficient multi-modal large language model utilizing mamba-2, 2024. URL https://arxiv.org/abs/2407.19832. arXiv:2407.19832.

[16] Max Jaderberg, Valentin Dalibard, Simon Osindero, et al. Population based training of neural networks. *arXiv preprint*, 2017. URL https://arxiv.org/abs/1711.09846. arXiv:1711.09846.

[17] Yuvaraj K. ag-news-distilbert, 2024. URL https://huggingface.co/YuvarajK-g25ait2054/ag-news-distilbert. Fine-tuned DistilBERT base model on the AG News dataset.

[18] Aaron Klein, Jacek Golebiowski, Xingchen Ma, Valerio Perrone, and Cédric Archambeau. Structural pruning of large language models via neural architecture search. In *AutoML Workshop*, 2023. URL https://www.amazon.science/publications/structural-pruning-of-large-language-models-via-neural-architecture-search.

[19] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2019. URL https://arxiv.org/abs/1909.11942. arXiv:1909.11942.

[20] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Benoît Sagot, Niklas Muennighoff, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, et al. Bloom: A 176b-parameter open-access multilingual language model, 2022. URL https://arxiv.org/abs/2211.05100. arXiv:2211.05100.

[21] Cheng Li and Yiming Yang. Multi-objective population based training. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

[22] Jun Li and Peng Zhao. Bossnas: Exploring hybrid cnn-transformers with block-wisely self-supervised neural architecture search, 2021. URL https://arxiv.org/abs/2103.12424. arXiv:2103.12424.

[23] Qingyuan Li, Bo Zhang, and Xiangxiang Chu. Eapruning: Evolutionary pruning for vision transformers and cnns, 2022. URL https://arxiv.org/abs/2210.00181. arXiv:2210.00181.

[24] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, et al. Jamba: A hybrid transformer–mamba language model, 2024. URL https://arxiv.org/abs/2403.19887. arXiv:2403.19887.

[25] Han Liu, Yixuan Wang, and Yaochu Jin. Evolution meets diffusion: Neural architecture generation with diffusion models, 2025. URL https://arxiv.org/abs/2504.17827. arXiv:2504.17827.

[26] Lei Liu, Gary G. Yen, and Zhenan He. Evolutionvit: Multi-objective evolutionary vision transformer pruning under resource constraints. *Information Sciences*, 2025.

[27] Matyáš Lorenc and Roman Neruda. Utilizing evolution strategies to train transformers in reinforcement learning, 2025. URL https://arxiv.org/abs/2501.13883. arXiv:2501.13883.

[28] Pedro Henrique Martins, João Alves, Patrick Fernandes, Nuno M. Guerreiro, Ricardo Rei, Amin Farajian, Mateusz Klimaszewski, Duarte M. Alves, José Pombal, Nicolas Boizard, Manuel Faysse, Pierre Colombo, François Yvon, Barry Haddow, José G. C. de Souza, Alexandra Birch, and André F. T. Martins. Eurollm-9b: Technical report, 2025. URL https://arxiv.org/abs/2506.04079. arXiv:2506.04079.

[29] PROPOR 2024 Shared Task Organizers. Propor 2024 shared task on automatic classification of scientific domains (fos), 2024. URL https://huggingface.co/datasets/ivosimoes/PROPOR_FOS_Classification. Dataset used for the scientific domain classification task; see the shared task overview at PROPOR 2024.

[30] Haohao Qu, Liangbo Ning, Rui An, Wenqi Fan, Tyler Derr, Hui Liu, Xin Xu, and Qing Li. A survey of mamba, 2024. URL https://arxiv.org/abs/2408.01129. arXiv:2408.01129.

[31] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *AAAI Conference on Artificial Intelligence*, 2019.

[32] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint*, 2017. URL https://arxiv.org/abs/1703.03864. arXiv:1703.03864.

[33] Oliver Sieberling, Denis Kuznedelev, Eldar Kurtic, and Dan Alistarh. Evopress: Accurate dynamic model compression via evolutionary search. In *ICML*, 2025.

[34] David So, Quoc Le, and Chen Liang. Evolved transformer. In *International Conference on Machine Learning (ICML)*, 2019.

[35] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.

[36] TechxGenus. Mini-Jamba, 2024. URL `https://huggingface.co/TechxGenus/Mini-Jamba`. A lightweight hybrid Mamba-Transformer language model, based on the Jamba architecture.

[37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models, 2023. URL `https://arxiv.org/abs/2302.13971`. arXiv:2302.13971.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *NeurIPS*, 2017.

[39] Chao Wang, Jiaxuan Zhao, Licheng Jiao, Lingling Li, Fang Liu, and Shuyuan Yang. When large language models meet evolutionary algorithms: Potential enhancements and challenges, 2025. URL `https://arxiv.org/abs/2401.10510`. arXiv:2401.10510.

[40] Rui Wang, Liang Chen, and Xiang Li. Neural architecture search for transformers: A survey. *ACM Computing Surveys*, 2022.

[41] Daan Wierstra, Tom Schaul, Jan Peters, and Jürgen Schmidhuber. Natural evolution strategies. *Journal of Machine Learning Research*, 15:949–980, 2014.

[42] Xin Xu and Peng Zhou. Efficient gradient-free neural architecture search. *Neural Networks*, 2020.

[43] Shangshang Yang, Xiaoshan Yu, Ye Tian, Xueming Yan, Haiping Ma, and Xingyi Zhang. Evolutionary neural architecture search for transformer in knowledge tracing. In *NeurIPS*, 2023.

[44] Kaiyue Lu Zhen Qin Weixuan Sun Jiacheng Xu Yiran Zhong Zexiang Liu, Dong Li. Neural architecture search on efficient transformers and beyond, 2022. URL `https://arxiv.org/abs/2207.13955`. arXiv:2207.13955.

[45] Wei Zhang, Yanan Wang, and Yaochu Jin. Multi-population evolutionary neural architecture search. *IEEE Transactions on Evolutionary Computation*, 2021.

[46] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NIPS)*, 2015.

[47] Hao Zhao, Ming Zhang, Wei Zhao, Peng Ding, Shuai Huang, and Dong Wang. Cobra: Extending mamba to multi-modal large language model for efficient inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. URL `https://ojs.aaai.org/index.php/AAAI/article/view/33131`.

[48] Peng Zhou and Xin Xu. Eg-nas: Efficient neural architecture search via evolutionary graph representation. In *NeurIPS*, 2019.

[49] Yifan Zhou, Ming Liu, and Hao Wang. Multi-objective evolutionary neural architecture search for medical image analysis using transformer and large language models in advancing public health. *Expert Systems with Applications*, 2024.

Appendices

Appendix A

Additional Experimental Data

This appendix provides the complete per-seed results for AG News and PROPOR, together with the PROPOR evolution plots.

A.1 Per-Seed AG News Results

Due to the width of the full table, we split it into two parts: Table A.1 lists the identification and training setup of each model, and Table A.2 reports the performance metrics.

Table A.1: AG News models – identification and training setup.

Model	**Params**	**Genotype**	**Seed**	**Steps,BS**
EvoModel1-E1	44.5M	[0,0,0,0,0]	42	100,16
EvoModel2-E1	44.5M	[0,0,0,0,0]	123	100,16
EvoModel3-E1	44.5M	[0,0,0,0,0]	999	100,16
EvoModel4-E1	44.5M	[0,0,0,0,0]	2024	100,16
EvoModel5-E1	43.4M	[0,0,0,0]	7	100,16
EvoModel6-E1	43.4M	[0,0,0,0]	2026	100,16
EvoModel7-E1	44.5M	[0,0,0,0,0]	22	100,16
E1-Resume	–	–	–	–
Mini-Jamba	69.5M	[0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1]	–	–
JambaGen	110M	[0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,0,0]	–	–
EvoModel1-E2	42.5M	[0,0,1,1]	42	400,16
EvoModel2-E2	43M	[0,0,0,1]	123	400,16
EvoModel3-E2	44M	[0,0,1,0,0]	999	400,16
EvoModel4-E2	42.1M	[1,1,1,0]	2024	400,16
EvoModel5-E2	43M	[0,0,0,1]	7	400,16
EvoModel6-E2	42.5M	[0,1,0,1]	2026	400,16
EvoModel7-E2	44M	[0,0,0,0,1]	22	400,16
E2-Resume	–	–	–	–
EvoModel1-E3	42.5M	[0,1,1,0]	42	300,16
EvoModel2-E3	43.6M	[0,0,0,1,1]	123	300,16
EvoModel3-E3	43M	[0,0,0,1]	999	300,16
EvoModel4-E3	42.5M	[0,0,1,1]	2024	300,16
EvoModel5-E3	43M	[0,0,0,1]	7	300,16
EvoModel6-E3	42.5M	[0,0,1,1]	2026	300,16
EvoModel7-E3	44M	[0,0,0,0,1]	22	300,16
E3-Resume	–	–	–	–

Table A.2: AG News models – performance metrics.

Model	**Time (h)**	**Lat (ms)**	**Acc**	**Prec**	**Rec**	**F1**
EvoModel1-E1	7.89	9.10	92.2	92.2	92.2	92.2
EvoModel2-E1	7.42	9.05	92.4	92.4	92.4	92.4
EvoModel3-E1	8.37	8.92	92.4	92.5	92.4	92.4
EvoModel4-E1	10.07	9.08	92.2	92.3	92.2	92.2
EvoModel5-E1	9.05	8.37	92.4	92.4	92.4	92.4
EvoModel6-E1	14.46	8.27	92.2	92.2	92.2	92.2
EvoModel7-E1	12.45	9.18	92.4	92.4	92.4	92.4
E1-Resume	9.96	8.85 (0.37)	92.31(0.11)	92.34(0.11)	92.31(0.11)	92.31(0.11)
Mini-Jamba	–	–	92.5	92.5	92.5	92.5
JambaGen	–	–	92.1	92.1	92.1	92.1
EvoModel1-E2	18.87	7.00	91.9	91.9	91.9	91.8
EvoModel2-E2	20.33	7.60	92.2	92.2	92.2	92.2
EvoModel3-E2	21.48	8.49	92.3	92.3	92.3	92.3
EvoModel4-E2	23.83	6.44	91.9	91.8	91.9	91.8
EvoModel5-E2	23.88	8.07	91.9	92.0	91.9	91.9
EvoModel6-E2	28.19	6.58	92.3	92.4	92.3	92.3
EvoModel7-E2	35.30	8.63	92.3	92.3	92.3	92.3
E2-Resume	24.55	7.54 (0.89)	92.11(0.20)	92.13(0.23)	92.11(0.20)	92.09(0.24)
EvoModel1-E3	20.00	7.55	92.1	92.1	92.1	92.1
EvoModel2-E3	22.18	8.15	91.9	91.9	91.9	91.9
EvoModel3-E3	22.37	8.18	92.1	92.1	92.1	92.1
EvoModel4-E3	23.46	7.51	92.1	92.0	92.1	92.1
EvoModel5-E3	24.93	7.73	92.2	92.2	92.2	92.2
EvoModel6-E3	24.44	6.17	91.9	91.9	91.9	91.9
EvoModel7-E3	26.98	8.68	92.2	92.2	92.2	92.2
E3-Resume	23.48	7.71 (0.80)	92.07(0.13)	92.06(0.13)	92.07(0.13)	92.07(0.13)

A.2 Per-Seed PROPOR Results

The PROPOR results are also split into two linked tables.

Table A.3: PROPOR models – identification and training setup.

Model	**Params**	**Genotype**	**Seed**	**Steps,BS**
EvoModel1-E1	43.4M	[0,0,0,0]	42	400,4
EvoModel2-E1	43.4M	[0,0,0,0]	123	400,4
EvoModel3-E1	43.4M	[0,0,0,0]	999	400,4
EvoModel4-E1	43.4M	[0,0,0,0]	2024	400,4
EvoModel5-E1	48.9M	[0,0,0,1,0,0,0]	7	400,4
EvoModel6-E1	44M	[0,0,0,1,0]	2026	400,4
EvoModel7-E1	43M	[0,0,1,0]	22	400,4
E1-Resume	–	–	–	–

Table A.4: PROPOR models – performance metrics.

Model	**Time (h)**	**Lat (ms)**	**Acc**	**Prec**	**Rec**	**F1**
EvoModel1-E1	36.83	8.67	89.1	84.9	85.2	85.0
EvoModel2-E1	37.93	8.63	89.0	85.4	84.2	84.7
EvoModel3-E1	37.97	8.87	89.2	84.7	85.4	85.0
EvoModel4-E1	101.13	9.02	89.2	85.2	84.0	84.4
EvoModel5-E1	93.36	13.32	87.0	82.9	81.4	82.1
EvoModel6-E1	47.98	9.11	87.1	82.4	81.9	82.2
EvoModel7-E1	49.13	8.07	86.1	81.1	80.3	80.7
E1-Resume	57.76	9.38 (1.77)	88.1	83.8	83.2	83.44

A.3 PROPOR Evolution Plots

For completeness, we include the fitness, depth, and layer-type proportion evolution plots for the PROPOR task (400 steps, batch size 4). The trends are similar to those observed in the AG News runs: the population rapidly converges to shallow, Mamba-dominated architectures.

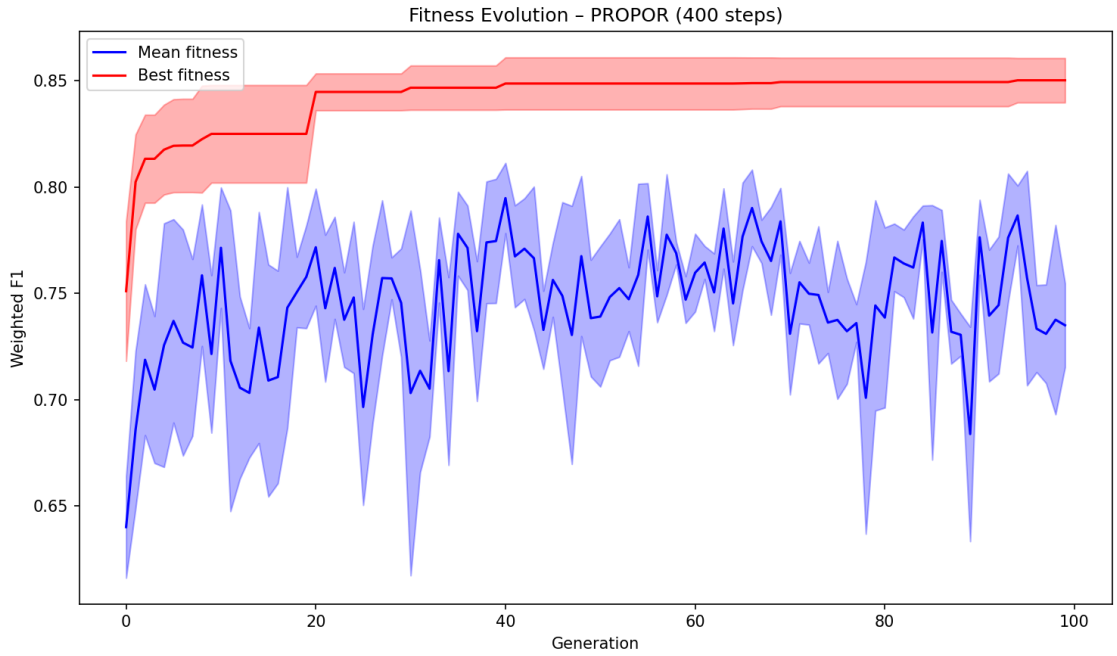

Figure A.1: Fitness evolution for PROPOR (400 steps).

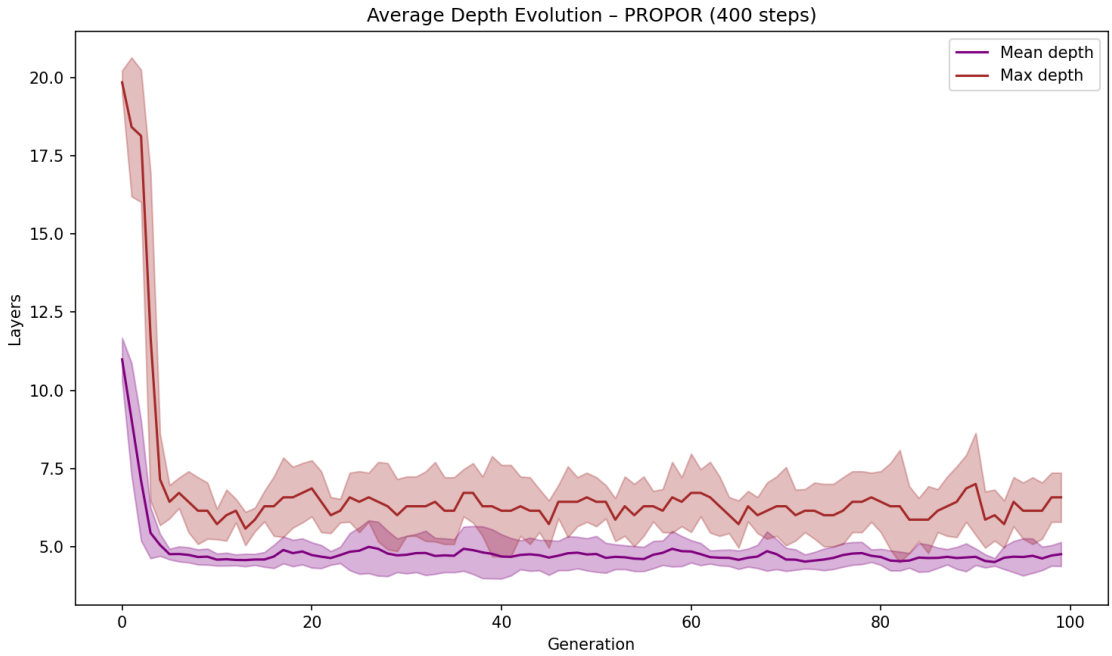

Figure A.2: Average depth evolution for PROPOR (400 steps).

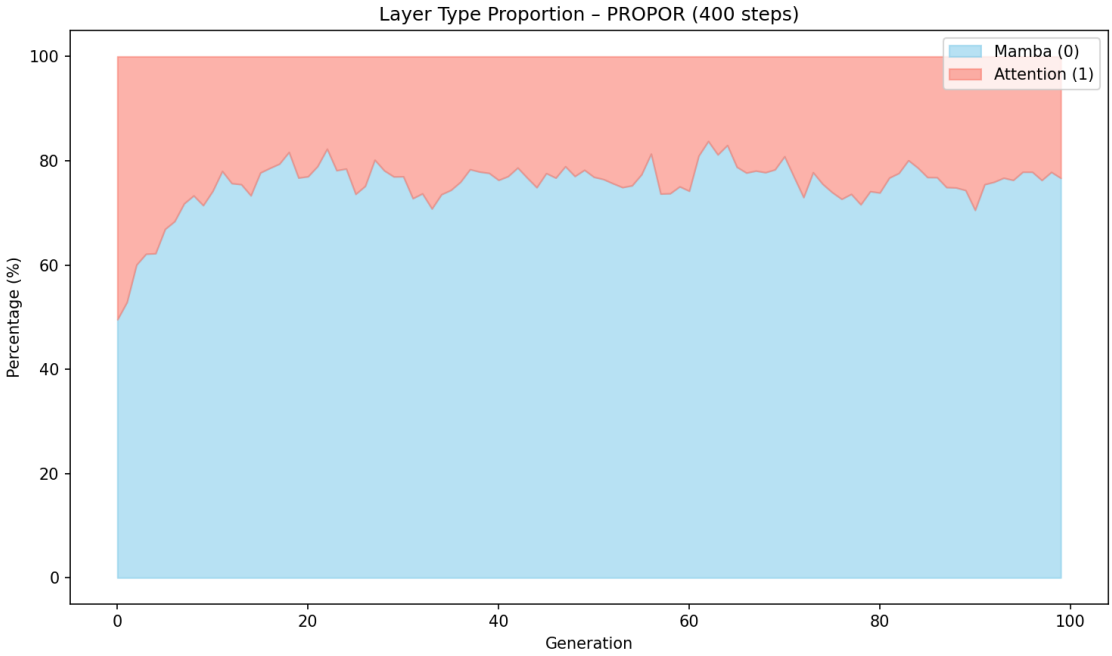

Figure A.3: Layer type proportion for PROPOR (400 steps).